Chapter 1. Language and Fundamentals of Programming

Q1. What is a Language?
Answer: A language refers to a system of communication used to write instructions that a machine, typically a computer, can understand and execute.

Q2. What are the different types of language?
Answer: Types of languages (based on understandability of human or system), there are two categories:

1. Natural Language
2. Programming Language

Natural Language:

- Used by humans for communication, such as English, Spanish, etc.
- Ambiguous and context-dependent.

Programming Language:

- A formal language used to write instructions for computers.
- Consists of a set of rules (syntax) and semantics that define how instructions are structured and interpreted.
- Programming languages enable the creation of algorithms and logic to solve problems, automate tasks, and process data.

Q3. What are the characteristics of Programming language
Answer: The characteristics of a Programming Language are:

- Syntax: Defines the rules for writing valid code (e.g., structure of statements, use of symbols).
- Semantics: Determines the meaning of the instructions written in the language.

Q4. What are the types of langauge as a developer you must have idea on?
Answer:

i. Low-Level Languages:
- Closer to machine code, hard to understand for humans.
- Examples: Assembly language, Machine code.
- Advantages: More control over hardware, faster execution.

ii. High-Level Languages:
- Closer to human language, easy to understand and use.
  Examples: Python, Java, C++, JavaScript.
- Platform-independent, easy to debug and maintain.

iii.Middle-Level Languages:
- Provide low-level access and high-level features (e.g., C).

Q5. Give some examples of popular programming languages.
Answer: For Web Development: JavaScript, HTML, CSS, PHP.
For Mobile App Development: Java, Kotlin, Swift.
For Game Development: C++, Unity (C#).

For Data Science: Python, R.
For System Programming: C, Rust.
***Java can be used in developing all the above given domains.

WHY TO LEARN PROGRAMMING LANGUAGE?

- Automates repetitive tasks.
- Solves complex problems.
- Builds innovative software and applications.
- High demand in the job market with lucrative career opportunities.

Q6. What are the types of Applications in Programming?

Answer: Applications are categorized based on their functionality, target audience, and platform. Here's an overview of the major types:

1. Desktop Applications

Description: Software installed and run on personal computers or laptops.
Examples: Microsoft Word, Adobe Photoshop, VLC Media Player.
Technologies Used:Java (Swing, JavaFX).
C# (.NET Framework).
Python (Tkinter, PyQt).
Advantages:
- Works offline.
- High performance on local machines.

Limitations:
Limited to the machine where it's installed.

2. Web Applications

Description: Applications accessed via web browsers over the internet.
Examples: Gmail, Facebook, Amazon.
Technologies Used:
- Frontend: HTML, CSS, JavaScript (React, Angular).
- Backend: Java (Spring), Python (Django), PHP.
- Databases: MySQL, MongoDB.

Advantages:
- Accessible from anywhere with internet.
- Easy updates and maintenance.

Limitations:
- Requires a stable internet connection.
- May have slower performance compared to desktop apps.

3. Mobile Applications

Description: Software designed specifically for mobile devices.
Types:
- Native Apps: Built for a specific platform (e.g., iOS or Android).
- Hybrid Apps: Use a single codebase for multiple platforms.

Examples: WhatsApp, Instagram, Uber.

Technologies Used:
- Native: Java/Kotlin (Android), Swift (iOS).
- Hybrid: React Native, Flutter.

Advantages:
- Portable and always available.
- Leverages mobile device features like GPS and camera.

Limitations:
Platform dependency for native apps.

4. Enterprise Applications
Description: Large-scale software designed to support business processes and workflows.
Examples: SAP ERP, Salesforce, Oracle Financials.
Technologies Used:
- Java (Spring, Java EE).
- .NET Framework.
- Python.

Advantages:
- Scalable and secure.
- Integrates multiple departments and processes.

Limitations:
- Expensive to develop and maintain.

5. Game Applications
Description: Interactive software designed for entertainment or educational purposes.
Examples: Call of Duty, Candy Crush, Minecraft.
Technologies Used:
- Unity (C#).
- Unreal Engine (C++).
- GameMaker Studio.

Advantages:
- High engagement and user interaction.
- Supports advanced graphics and real-time gameplay.

Limitations:
- Requires high-performance hardware.

6. Embedded Applications
Description: Software designed for specific hardware devices.
Examples: Software in washing machines, smart TVs, and IoT devices.
Technologies Used:
- C, C++.
- Python (for IoT).

Advantages:
- Optimized for specific devices.
- Low resource consumption.

Limitations:
- Limited functionality compared to general-purpose applications.

7. Cloud-Based Applications
Description: Applications hosted on the cloud and accessed via the internet.
Examples: Google Drive, Dropbox, Microsoft Azure.
Technologies Used:
- Backend: Node.js, Java, Python.

Cloud Platforms: AWS, Google Cloud, Microsoft Azure.
Advantages:
- Accessible from any device.
- Scalable and cost-efficient.

Limitations:
- Dependence on internet connectivity.

8. Command-Line Applications
Description: Applications that run in a command-line interface (CLI) and take input/output through text commands.

Examples: Git, Linux terminal commands, Python scripts.
Technologies Used:
- Python, Bash scripting, C++.
Advantages:
- Lightweight and fast.
- Ideal for developers and system administrators.
Limitations:
- Steep learning curve for non-technical users.

9. Scientific and Data Applications
Description: Software used for research, analysis, and simulations.
Examples: MATLAB, NumPy, SciPy.
Technologies Used:
- Python (Pandas, NumPy, Matplotlib).
- R, MATLAB.
Advantages:
- High accuracy and efficiency for complex calculations.
- Extensively used in academia and research.
Limitations:
- Requires domain expertise.

*** **TYPES OF APPLICATIONS BASED ON ITS EXECUTION:**
1. Stand Alone Applications
2. Web Based Applications

Stand-Alone Applications:

A stand-alone application is a software program that runs independently on a single device or system without requiring a network connection or additional support from external systems. These applications are self-contained and provide all the functionality locally.

Characteristics of Stand-Alone Applications:

- Local Execution: Installed and executed directly on the user's device.
- Offline Functionality: Does not require an internet connection to function.
- Platform-Specific: Often designed for specific operating systems like Windows, macOS, or Linux.

Examples:
- Media players (e.g., VLC Media Player).
- Word processors (e.g., Microsoft Word).
- Graphic design tools (e.g., Adobe Photoshop).

Advantages:
- Reliable in environments without internet access.
- High performance since it does not depend on external servers.
- Greater control over data security as the data is stored locally.

Disadvantages:
- Limited to the device where it is installed.
- Updating or sharing data across devices can be cumbersome.
- Can consume significant local storage and resources.

Internet Applications

An internet application is software that relies on a network connection to function and is typically hosted on a web server. Users access these applications through web browsers or dedicated clients.

Characteristics of Internet Applications:

- Network Dependency: Requires an active internet connection for most or all of its functionality.
- Platform Independence: Can run on any device with a compatible web browser.
- Client-Server Model: Processes are divided between the user's device (client) and a remote server.

Examples:
- Email services (e.g., Gmail).
- E-commerce platforms (e.g., Amazon).
- Social media (e.g., Facebook, Instagram).

Advantages:
- Accessible from any device with an internet connection.
- Easy to update and maintain since updates are applied to the server.
- Data is centrally stored, enabling real-time sharing and collaboration.

Disadvantages:
- Requires a stable internet connection.
- May face security risks if not properly protected.
- Performance depends on server response and network speed.

Comparison: Stand-Alone vs. Internet Applications

Feature	Stand-Alone Applications	Internet Applications
- Connectivity	Works offline	Requires internet connection
- Platform	Device-specific	Cross-platform via browsers
- Maintenance	Manual updates by users	Automatic updates from the server
- Data Storage	Local	Centralized on a server
- Examples	VLC, Photoshop, MS Word	Gmail, Amazon, YouTube

Q. WHAT IS JAVA?
Answer:
Java is a simple, open source, high-level, object-oriented,platform-independent programming language developed by Sun Microsystems (now owned by Oracle Corporation) and released in 1995.It is widely used for building cross-platform applications,ranging from desktop software to enterprise-level solutions and Android mobile apps.

Q. What are the features of Java?
Answer: Features of Java:
-mainly its open source i.e., its free of cost to use java to develop applications :-).

Till Java 10 it was completely free of cost but, from Java 11 onwards its both open source as well as licensed.
Now we have:
i. Open JDK
ii. Oracle JDK

1. **Simple:**Its easy and simple to learn and develop projects. Because all the concepts used in java are simple as the

syntaxes used in java are easy to learn.
It doesnot use lots of confusing concepts
such as Pointers(used in C),multiple inheritance(C++) etc.
And above to all, it has lots of predefined libraries due to which
development of project becomes faster and easy.

2. Secured: There is a special feature inbuilt i.e., Byte Code Verifier in JVM. Its responsibility is to check whether the code contains any malicious code or not. If it finds anything suspicious then the code is not allowed to be exceuted.Due to this the system is safe.

It also provides access modifier 'private' , which when used with an element, the element will not be allowed to access outside the class.

3. Object Oriented: Java is a class based program which means every program will be written by starting the program with class as the programs in java are basically for representing real world objects in programming world.And its possible using class.A class represents the blueprint of objects.

4. Byte Coded: Java programs are compiled to achieve platform independent type apps.

5. Interpreted: Byte codes are generally dependent on the current OS but interpreter takes it and generates native code for OS to perform action on commands present in Byte codes.
AS like java other PL are also interpreted PL.such as
C#.NET, python, javascript, PHP etc.

6. Portable(Platfrom Independent): Application of java developed in one platform can work in other platforms also.We dont have to rewrite the complete source code, we just need the byte code to be shared.

7. MultiThreaded: It supports to develop applications which can handle multiple tasks parallely at a time.In java each task can have its separate thread to complete its execution independent of any other tasks.

8. Robust(Strong): It has a fail safe system, which means whenever an abnormal or unexpected condition is arised then the program terminates only after saving the given data and there is no loss of data.

C is not type checked, but java is strictly type checked.
You can't have loss of data.
Exception handling will support in making the application continue the execution even after any exception may arise.

9.High Performance: Java program execution is faster because of multithreading and a special feature from Java 1.3 i.e JIT Compiler.

10.Distributed: We can develop applications which can be accessible through worldwide remotely. It have special technologies such as :
RMI,XML,EJB,WebServices,REST API,JSONs.....to be conitnued in Adv.java

11. Dynamic: We can add new features to application without affecting the old features.

Beacuse of OOP concepts:

1. inheritance
2. encapsulation
3. abstraction
4. polymorphism.

12.Architecture Neutral: Whatever result you get in one platform the same you can get in another platform also(memory size is fixed for all OS and processors).
In C 'int' has different size from one OS to another OS.
but in java its always same in all OS.

13. Garbage Collected: It manages the memory by clearing the unused objects. We dont have to destroy objects, they are automatically destroyed by JVM by using one of its resource called as Garbage Collector.

Q1. Which feature is necessary in a technology to develop Internet Applications?
Answer: Technology must have characteristic of developing platform independent application.

Q2. How did Java achieve platform independent feature?
Answer: By allowing to shift the Machine Language from compilation phase to execution phase by the help of generating bytecodes.JVM converts the bytecode into current OS understandable machine language .

Q3.Java is platform dependent or independent?
Answer: Independent

Q4. JVM is platform dependent or independent?
Answer: Dependent because for every OS we have separate JVM.

Q5. What are the types of Java software?
Answer: Java software is divided into two types:

1. JDK:it has both compiler and JVM
2. JRE:it has only JVM

JDK and JRE both are principle products of Java.

Q6. Can we execute a java program using using lower version JVM than compiler version?
Answer: No.

Q7. What type of applications can be developed using Java?
Answer: We can develop following applications:

1. Desktop Applications
2. Web Applications
3. Enterprise Applications
4. Interoperable Applications
5. Mobile Applications
6. Gaming Applications
7. Automation Testing Applications
8. iOT, Cloud Computing, Big Data Hadoop Applications
9. Data Science & Data Analytics Applications
10. ML, Deep Learning, AI

Q8. What are the key Components of Java?

Answer: Java Development Kit (JDK): A toolkit that includes tools for developing Java applications,
such as the compiler (javac), debugger, and libraries.
Java Virtual Machine (JVM): Executes Java bytecode on the host machine,making Java platform-independent.
Java Runtime Environment (JRE): Provides the libraries and JVM required to run Java programs.

Q9. Give some advantages of Java?

Answer: Advantages of Java:
- Portable and platform-independent.
- Extensive standard library (Java API).
- Backed by a large community and continuous updates.
- Versatile and widely adopted across industries.

Java has become a cornerstone in modern software development due to its reliability, scalability, and versatility.

Q10. How Java Achieves Platform Independence:

Answer:

1.Compilation into Bytecode:

When a Java program is compiled using the javac compiler, it is not directly translated into machine code (specific to a computer's hardware).
Instead, the source code is converted into an intermediate form called bytecode (contained in .class files).
Bytecode is a platform-independent, low-level representation of the program.

2. Java Virtual Machine (JVM):

The JVM is a virtual engine that interprets the bytecode and executes it on the host machine.
Each platform (Windows, macOS, Linux, etc.) has its own JVM implementation tailored for that
operating system.
However, the bytecode remains the same, making the Java program "write once, run anywhere" (WORA).

3. Execution on Any Platform:

Since the JVM is platform-specific, it acts as a mediator between the bytecode and the underlying
operating system.
As long as a compatible JVM is installed, the same .class file can be executed on any machine,
regardless of the OS or hardware.

Key Components Enabling Platform Independence:

Bytecode:

Acts as a universal language understood by all JVMs, ensuring the compiled program is not tied to any specific platform.

Java Runtime Environment (JRE):

Includes the JVM and essential libraries needed to execute a Java program, ensuring uniform behavior across platforms.

Abstracted APIs:

Java provides standard APIs for operations (like file I/O, network communication) that internally manage platform-specific differences.

Q11. Why is Bytecode Platform-Independent, but JVM is Not?
Answer: Bytecode: It is standardized and the same across all platforms, making it platform-independent.
JVM: Since it interacts with the host operating system and hardware, each platform requires its own JVM implementation.
Example of Platform Independence in Action:
Write and compile a Java program on a Windows machine:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Compilation (javac HelloWorld.java) produces HelloWorld.class.
Transfer the HelloWorld.class file to a Linux or macOS system.
Execute the program on the target system (java HelloWorld):
The JVM on that platform interprets the bytecode and runs the program, outputting:
Hello, World!

See Java's platform independence is rooted in its bytecode, interpreted by a platform-specific JVM, enabling the same compiled program to run seamlessly across different
systems. This design ensures portability, making Java one of the most versatile programming
languages.

Q12. Explain differences between JDK,JVM and JRE.
Answer: 1. Java Development Kit (JDK)
Purpose: The JDK is the complete development environment for Java programming. It provides everything a developer needs to write, compile, and debug Java applications.

Components:
- JVM: The core engine that runs Java bytecode.
- JRE: The runtime environment needed for the JVM to run.
- Compiler (javac): Converts Java source code (.java files) into bytecode (.class files), which can be executed by the JVM.
- Debuggers: Tools like jdb (Java Debugger) for troubleshooting Java programs.
- Java APIs: Pre-written classes and libraries to help developers implement various functionalities (e.g., handling I/O, networking, GUI development, etc.).

Use Case: Developers use the JDK to create Java applications, as it includes the tools required to develop and compile Java code. Without the JDK, you cannot compile Java code or use development tools.
Example: If you're building a Java application that performs database operations, you need the JDK to write the code, compile it, and use the libraries (such as java.sql).

2. Java Virtual Machine (JVM)

Purpose: The JVM is the execution engine of Java. It translates compiled bytecode into machine
code that a specific operating system or hardware can understand. The JVM is the core of the
"write once, run anywhere" (WORA) capability of Java.

How It Works:
- Bytecode Interpretation: Java source code is compiled into bytecode, which is not specific to any platform. The JVM reads the bytecode and converts it into machine-specific instructions.

- Memory Management: The JVM is responsible for managing memory through the heap (where objects are stored) and stack (where method calls and local variables are stored).

- Garbage Collection: The JVM automatically handles memory management by reclaiming memory from
objects that are no longer in use (garbage collection).

- Platform Dependency: There is a separate JVM for each platform (Windows, Linux, macOS),
but all JVMs can execute the same bytecode.
Use Case: The JVM allows Java programs to be executed on any platform as long as the appropriate
JVM is installed. You do not need to worry about the underlying operating system when writing
Java code.
Example: If you have a Java program compiled into a .class file, the JVM is what actually runs that program on your machine, regardless of whether you're using Windows, Linux, or macOS.

3. Java Runtime Environment (JRE)
Purpose: The JRE provides the necessary environment to run Java programs. It includes the JVM and other libraries needed for Java applications to execute. However, it does not include
development tools like compilers.

Components:
- JVM: The execution engine.
- Libraries: The set of predefined classes and APIs necessary for running Java applications.
These include essential libraries for handling tasks such as file I/O, networking, and data manipulation.
- Supporting Files: Configuration files and resources needed by the JVM.
Use Case: The JRE is all that is needed for running Java applications. If you are an end-user who just wants to run a Java application, you only need the JRE, not the full JDK.

Example: If you're installing a Java-based game or application on your system, you need the JRE (and JVM) to run it. However, if you're a developer creating the game, you'd need the JDK.

Relationship Between JDK, JVM, and JRE

JDK is used by developers to create Java applications. It includes the JRE
(which includes the JVM) along with development tools such as compilers and debuggers.

JRE is used by end-users to run Java applications. It contains the JVM, which is responsible for running the bytecode, and the required libraries to support the application's execution.

JVM is the core engine that runs Java bytecode. It is included in both the JDK and the JRE. Without the JVM, Java bytecode cannot be executed.

Visualizing the Structure:

JDK (Developer's Toolkit)
├── JVM (Executes bytecode)
├── JRE (Runtime environment for Java applications)
│ ├── JVM
│ └── Java Libraries (Support classes for execution)
├── Development Tools (Compiler, Debugger, etc.)
├── Java APIs (Standard libraries for development)

JDK: Contains everything for development (JVM + JRE + tools).
JRE: Contains everything for running Java applications (JVM + Java libraries).
JVM: The engine that executes the bytecode.
Summary:
JDK: A complete development kit that includes the tools to write, compile, and debug Java programs. It includes the JRE and the JVM.
JRE: A runtime environment that provides the necessary components to run Java applications
(includes the JVM and supporting libraries).
JVM: The engine that runs Java bytecode, allowing Java programs to be platform-independent.

Q13. Explain the different ions of Java?
Answer: Java has several ions designed for different types of applications, ranging from desktop applications to large-scale enterprise systems, as well as mobile and embedded devices. These ions provide the necessary libraries and tools to cater to the specific requirements of the target environment.

Here are the main ions of Java:
1. Java Standard ion (Java SE):

Purpose:Java SE is the core platform for developing general-purpose Java applications. It provides the essential features and libraries required to build a wide range of applications, from simple desktop applications to more complex systems.

Key Features:

- Core Libraries: Includes core Java libraries like collections, I/O, networking, utilities, and concurrency.
- JVM: Provides the Java Virtual Machine (JVM) to run Java bytecode on various platforms.
- Java APIs: Provides essential APIs for GUI development (Swing, AWT), file handling,
- networking, and database access (JDBC).

Tools: Includes tools like javac (Java compiler) and java (runtime environment).
Use Cases:Desktop applications (using Swing or JavaFX).
- Command-line tools.
- Standalone Java applications.
- Simple server-side applications.

Example: A basic console application or desktop application using Swing for the user interface.

2. Java Enterprise ion (Java EE) (Now called Jakarta EE)

Purpose: Java EE (now known as Jakarta EE after being transferred to the Eclipse Foundation)
is designed for building large-scale, distributed, and multi-tiered enterprise applications. It extends Java SE with additional APIs and services for web, enterprise, and distributed applications.

Key Features:
- Enterprise APIs: Includes APIs for web services (JAX-RS, JAX-WS), servlets (Servlets, JSP),
- EJB (Enterprise JavaBeans), JMS (Java Message Service), and JPA (Java Persistence API)
  for managing relational data.
- Application Server: Typically deployed on application servers like Apache Tomcat, JBoss, or GlassFish.
- Scalability & Transactions: Supports enterprise-level services such as transaction management,
  security, messaging, and persistence.
- Cloud Compatibility: Jakarta EE applications are designed to be deployed on cloud-based
  infrastructure.

Use Cases:
- Enterprise web applications (e.g., customer relationship management systems, e-commerce platforms).
- Distributed systems (such as microservices, enterprise messaging).
- Enterprise-grade back-end services for high-availability systems.

Example: A web-based banking application or a cloud-based microservices application.

3. Java Micro ion (Java ME)

Purpose: Java ME is designed for developing applications on mobile and embedded devices.
It is a lightweight version of Java that targets devices with limited resources, such as mobile phones, IoT devices, and other embedded systems.

Key Features:
- Profile & Configuration: Java ME defines profiles and configurations based on the capabilities of different devices, such as Mobile Information Device Profile (MIDP) and Connected Limited Device Configuration (CLDC).
- Limited Libraries: Provides essential libraries for mobile and embedded devices, such as GUI development, networking, and storage.
- Small Footprint: Optimized for devices with limited memory, processing power, and storage capacity.
- Device-Specific APIs: Includes APIs tailored for mobile devices (e.g., Java ME CLDC for devices with limited resources, MIDP for mobile applications).

Use Cases:
- Mobile applications (especially older mobile devices before the rise of Android and iOS).
- Embedded devices like printers, set-top boxes, and smart TVs.

- IoT devices with limited computational resources.

Example: A mobile application running on an old Nokia feature phone using Java ME or a smart sensor in an IoT network.

4. JavaFX (Client-Side Java)

Purpose: JavaFX is a platform for creating rich, interactive desktop applications with sophisticated user interfaces. It can be used for developing both 2D and 3D applications, and is meant to replace Swing for building GUI applications in Java.

Key Features:

- Rich UI Components: Provides modern UI controls, charts, and graphics libraries.
- FXML: A markup language to define UI structure (similar to HTML).
- Scene Graph: JavaFX uses a scene graph to represent all the graphical objects.
- Integrated with Java SE: JavaFX is bundled with Java SE, making it easy to integrate with standard Java applications.

Use Cases:

- Desktop applications with rich user interfaces.
- Media applications that need to display video or handle 3D graphics.

Example: A Java-based video streaming application or interactive visualization tool.

5. Java Card

Purpose: Java Card is a version of Java designed for smart cards and other small devices with constrained resources. It provides a secure environment for executing applications, often used in banking cards, SIM cards, and embedded secure devices.

Key Features:

- Small Footprint: Optimized for small, resource-constrained devices.
- Security Features: Supports secure transactions and cryptographic operations for applications like payment systems and identification.
- Smart Card Support: Specifically designed for smart card applications.

Use Cases:

- Banking: Secure payments using smart cards.
- Identity Verification: SIM cards and other personal identification systems.
- Embedded Secure Devices: Devices requiring high security with low resource consumption.

Example: SIM card applications, payment cards, and ID cards.

Summary of Java ions:

Conclusion:

- Java SE is ideal for general-purpose applications.
- Java EE (Jakarta EE) is designed for building large-scale enterprise solutions.
- Java ME targets mobile and embedded devices with limited resources.
- JavaFX focuses on rich client-side applications with modern UIs.
- Java Card is for secure applications on smart cards and embedded devices.

Each ion is tailored for specific use cases, and understanding these distinctions helps developers choose the right tool for the job.

Q14. Explain different versions of Java.

Answer:

Java has evolved significantly since its inception in 1995. Each version of Java has brought new features, improvements, and optimizations. Here's an overview of the major versions of Java, their features, and improvements over the years:

1. Java 1.0 (1996)

First release of Java, with the promise of "Write Once, Run Anywhere" (WORA).
Key Features:

- Basic language features: Classes, objects, methods, primitive types, and inheritance.
- Standard Libraries: Core libraries for input/output, data structures, and networking.
- JVM: Java Virtual Machine (JVM) was introduced to execute Java bytecode on various platforms.

2. Java 1.1 (1997)

Introduced key improvements in performance and the addition of several new features.
Key Features:

- Inner classes: Allowed defining classes within other classes.
- JavaBeans: A framework for reusable software components.
- JDBC (Java Database Connectivity): Introduced to connect Java applications to relational databases.
- RMI (Remote Method Invocation): For remote communication between objects across a network.

3. Java 1.2 (1998) - "Java 2"

Marked a major shift with the introduction of Java 2 and a new versioning system.
Key Features:

- Swing: A new GUI toolkit to replace AWT, offering more modern and customizable user interfaces.
- Collections Framework: Unified framework for storing and manipulating data.
- Java Plug-in: To allow Java applications to run in web browsers.
- J2EE (Java 2 Enterprise ion): Introduced for building enterprise applications.

4. Java 1.3 (2000)

Focused on performance improvements, scalability, and robustness.
Key Features:

- J2SE: Focused on improvements to core libraries and the JVM.
- HotSpot JVM: Introduced for enhanced performance through Just-In-Time (JIT) compilation and garbage collection optimizations.
- JNDI (Java Naming and Directory Interface): For accessing directory services like LDAP.

5. Java 1.4 (2002)

Introduced important features aimed at improving networking, I/O, and XML handling.
Key Features:

- Regular Expressions: Support for regular expressions (regex) in Java.
- NIO (New I/O): Introduced new I/O libraries for better performance and scalability.
- Logging API: A standard logging API for managing logs.
- Assertions: Introduced assertions for debugging and testing purposes.

6. Java 5 (2004) - "J2SE 5.0" (also known as Java 1.5)

A major update, featuring many new language enhancements.
Key Features:
- Generics: To allow parameterized types, improving type safety and reusability.
- Metadata Annotations: Introduced annotations to provide metadata about classes, methods, and fields.
- Enumerated Types: A type-safe way of defining a fixed set of constants.
- Varargs: Allowed methods to accept a variable number of arguments.
- Enhanced for-loop: A new for-loop to iterate through collections and arrays.

7. Java 6 (2006)

Focused on performance improvements, as well as enhancements to existing features.
Key Features:
- Scripting support (JSR 223): Introduced scripting capabilities via the Java Compiler API and integration with JavaScript through the Java Compiler API and Scripting API.
- Java Compiler API: Allowed developers to compile Java code dynamically.
- Performance Enhancements: Improvements in the JVM, garbage collection, and the compiler.
- Web Services (JAX-WS): Simplified development of web services.

8. Java 7 (2011)

Brought several new features for simplifying code and improving efficiency.
Key Features:
- Try-with-resources: Simplified resource management, making it easier to handle closeable resources like files and sockets.
- Strings in Switch Statements: Support for using String in switch statements.
- NIO 2: Enhanced NIO (New I/O) with better file system and path manipulation support.
- Binary Literals: Support for representing binary values directly in code.
- Improved Type Inference: Enhancements to generics and type inference.

9. Java 8 (2014)

One of the most significant releases, introducing functional programming features to Java.
Key Features:
- Lambda Expressions: A functional programming feature to enable passing behavior as parameters (closures).
- Streams API: A powerful new API for processing sequences of elements (collections) in a functional style.
- Default Methods: Allowed methods in interfaces to have default implementations.
- New Date and Time API: A new java.time package for handling dates and times, addressing the flaws of the old Date and Calendar classes.
- CompletableFuture: A framework for asynchronous programming.

10. Java 9 (2017)

- Modular System: Introduced the Java Platform Module System (JPMS) to modularize the JDK and make applications easier to scale.

Key Features:

- Module System (Project Jigsaw): Modularized the JDK, allowing developers to create modular
applications.
- JShell: A command-line REPL (Read-Eval-Print Loop) for interactive Java programming.
- Stack-Walking API: Improved diagnostic features for inspecting the JVM stack.

11. Java 10 (2018)

Introduced incremental improvements and focused on enhancing the overall developer experience.
Key Features:
- Local-Variable Type Inference (var): Introduced var for local variable type inference.
- G1 Garbage Collector improvements for better performance.
- Application Class-Data Sharing: Improved JVM startup time by sharing class data between
applications.

12. Java 11 (2018)

The first Long-Term Support (LTS) release after Java 8.
Key Features:
- HttpClient API: Standardized API for making HTTP requests.
- Flight Recorder: A tool for collecting diagnostic data.
- Epsilon GC: A no-op garbage collector designed for performance testing.
- Removed Deprecated Features: Removal of outdated and deprecated features
(e.g., Applet API, Java Web Start).
- String Methods: New methods like isBlank(), lines(), and strip() to make String
manipulation easier.

13. Java 12 - 15 (2019-2020)

These versions introduced several feature enhancements and experimental features.
Key Features:
- JEP 334: JVM support for Records (introduced in Java 14).
- Switch Expressions (Java 12): Allowed switch to return a value and used ->
for cleaner syntax.
- Text Blocks (Java 13): Multi-line strings were simplified using triple quotes.
- JEP 358: Helpful NullPointerExceptions (improved error messages).
- Java 14: Preview of Records and Pattern Matching.

14. Java 16-17 (2021)

Java 16 introduced project Panama and Records as standard, while Java 17 was another LTS release.
Key Features:
- Project Loom: The introduction of virtual threads for simplifying concurrency.
- JEP 395: Strong encapsulation for internal APIs.
- Java 17 (LTS): Long-Term Support with several improvements like sealed classes (which allow you to define restricted class hierarchies), Pattern Matching, and Enhanced Switch Statements.

15. Java 18 and Beyond (2022-Present)

Java continues to evolve, with regular feature releases every six months.
Key features in recent versions include project Loom, project Panama, and continued

improvements in pattern matching and records.

Q15. What is Platform Independent application?
Answer: Platform independence means that a program or software can run on different operating systems or environments without requiring modification to its source code.

Q16. What is byte code?
Answer: Bytecode in Java is an intermediate, low-level representation of the source code that is platform-independent and designed to be executed by the Java Virtual Machine (JVM).
When you compile Java source code using the javac compiler, it is translated into bytecode, which is stored in .class files created by the compiler only if there is no error found during the compilation of .java file.

Q17. Why is byte code given to JVM for execution?
Answer: Bytecode is not directly executable by the operating system or hardware.
The JVM interprets or compiles the bytecode into machine code that is specific to the underlying platform, enabling Java's Write Once, Run Anywhere (WORA) principle.
Each platform (Windows, Linux, macOS, etc.) has its unique machine code instructions. Instead
of creating separate versions of a program for every platform, Java compiles the code to bytecode, which the JVM translates into platform-specific machine code at runtime.

Q18. What is byte code verifier?
Answer: The Bytecode Verifier is a key component of the Java Virtual Machine (JVM) that ensures
the integrity, safety, and validity of Java bytecode before execution. Its main role is to verify that the bytecode adheres to Java's security and correctness rules, preventing potentially harmful or incorrect code from being executed.Java programs can be distributed across networks or obtained from untrusted sources. Without proper verification, malicious or incorrectly compiled bytecode could:

- Corrupt memory.
- Violate Java's strict security model.
- Cause unpredictable behavior.

The Bytecode Verifier ensures that the bytecode is safe to execute, maintaining the robustness
and reliability of the Java platform.

Q19. When Does Bytecode Verification Happen?
Answer: During Class Loading:

- When the JVM loads a .class file, the Bytecode Verifier checks the validity of the bytecode.It ensures the bytecode is consistent with Java's language rules and does not perform illegal operations.

Q20. What the Bytecode Verifier checks?
Answer:
Structure and Format Validity:

Ensures the .class file adheres to the correct structure defined by the Java Class File Format.
Example: The file contains valid magic numbers, version information, constant pool data, etc.

Type Safety:

Verifies that all variable assignments, method calls, and type casts are legal.
Example: Ensures an int is not treated as a float without explicit casting.

Stack and Heap Usage:

Checks that the operand stack has the correct number of values and types for each operation.
Prevents stack overflow or underflow issues.
Ensures proper allocation and deallocation of objects on the heap.

Access Control:

Verifies that the bytecode does not illegally access private fields or methods of a class.
Ensures the visibility rules of public, protected, private, and default access modifiers are followed.

Code Validity:

Ensures that there are no:
- Illegal jumps or branches (e.g., jumping to the middle of an instruction).
- Invalid method signatures or class hierarchies.
- Violations of final classes or methods (e.g., attempting to override final methods).

Initialization Rules:

Ensures that all objects and variables are properly initialized before they are used.
Example: Verifies that a constructor has been called for an object before accessing its fields.

Q21. What are the advantages of Bytecode Verifier?
Answer: Security: Prevents malicious code from compromising system integrity.
Robustness: Ensures the reliability and correctness of Java applications.
Portability: Verifies bytecode regardless of the platform, contributing to Java's "Write Once, Run Anywhere" promise.
Error Prevention: Catches many potential issues at an early stage, before execution.

Q22. What is JIT Compiler?
Answer: The JIT (Just-In-Time) Compiler is a component of the Java Virtual Machine (JVM) that improves the performance of Java applications by converting bytecode into machine code
at runtime. This machine code is then executed directly by the processor, making execution much faster compared to interpreting bytecode line by line.

Q23. How JIT Compiler works?
Answer:
Initial Execution:

When a Java program starts, the JVM begins interpreting the bytecode.
As the program runs, the JVM identifies "hotspots" (frequently executed sections of code) using profiling information.

Compilation of Hotspots:

The JIT Compiler translates these hotspots from bytecode to native machine code.
Once compiled, the JVM executes the native code instead of interpreting the bytecode,

significantly improving performance.

Optimization:

The JIT Compiler applies various optimizations to improve the efficiency of the compiled code.

Q24. Difference between JIT Compiler and Interpreter.
Answer: JIT Compiler vs Interpreter

Feature	JIT Compiler	Interpreter
- Execution	Compiles bytecode to native code	Interprets bytecode line-by-line
- Performance	Faster for frequently executed code	Slower overall
- Startup Time	Slower (due to compilation overhead)	Faster
- Optimization	Runtime optimizations applied	None

====== Chapter 2. TOKENS IN JAVA ========

Tokens are the basic building blocks of a Java program.
They are the smallest individual units of a program that have meaning to the compiler.
They are used to represent the various elements of a program, such as keywords, identifiers, operators, and literals.

Tokens are classified into five types:

- **Keywords**
- **Identifiers**
- **Literals**
- **Punctuators**
- **Operators**

Q1. What is a keyword?
Answer: A special predefined word available in java which has special meaning for the system is called as keyword.
In java we have two types of keywords:

1. **Reserved Keywords (51)**
2. **Contextual Keywords (17)**

List of 51 Reserved Keywords:

abstract	**continue**	**for**	**new**	**switch**
assert	**default**	**goto**	**package**	**synchronized**
boolean	**do**	**if**	**private**	**this**
break	**double**	**implements**	**protected**	**throw**
byte	**else**	**import**	**public**	**throws**
case	**enum**	**instanceof**	**return**	**transient**
catch	**extends**	**int**	**short**	**try**
char	**final**	**interface**	**static**	**void**
class	**finally**	**long**	**strictfp**	**volatile**
const	**float**	**native**	**super**	**while**

Note:

Since Java 9, the underscore character is a reserved keyword
underscore (_) can't be used as an identifier or variable name.
---
* **goto** is not in use
* **const** is not in use

Upto JDK 8, there were 50 keywords but from JDK 9 its 51 just because of underscore added into the list of keywords.
As per the recent version of java 21, there are 51 reserved keywords whereas 17 contextual keywords.

About underscore in java:-

- Underscores in numeric literals are allowed .
- This feature allows you to separate groups of digits in numeric literals, which can improve the readability of your code. You can place underscores

only between digits, and you can use any number of underscores.

For example, the following code is all valid:
int value = 1_000_000;
long value = 1_000_000_000L;
float value = 3.14159_26F;
double value = 3.14159_26;

Using underscores in numeric literals is optional, but it can be helpful for making your code more readable, especially when working with large numbers.

Q2. What is an identifier?
Answer:

- Identifiers are names that are used to identify Classes,Interfaces,Methods,Variables,Constructors,Packages,Constants.
- They play a crucial role in giving meaning and structure to your code.
- They must start with a letter or an underscore, and can contain letters, numbers, underscores, and dollar.
- Identifiers cannot start with a digit and identifiers cannot be a Java keyword.
- Identifiers doesnot contain whitespace.

We can also say that an identifier is the name assigned to a part of the Java program. It helps to identify different elements uniquely in the code.

e. g:

```
class Student{
public static void main(String []args){
int rollNo = 10;
String name = "Jagannath";
}
}
```

Here in the above example Student,main,args,rollNo,name are identifiers.

Points to remember:

1. Always start identifier with a letter, or $

Invalid:
int 1score = 100; // Error: Cannot start with a digit

Valid:
int score1 = 100; // Starts with a letter
int _marks = 95; // Starts with underscore
int $salary = 50000; // Starts with dollar sign

Tip: Avoid using $ unless it's for autogenerated names (like inner classes).
_ is allowed but should be used with purpose (like _temp).
e.g:

2. Use camelCase for variables and methods

camelCase: First word is lowercase, subsequent words start with uppercase.
Example:

```
public class Student {
   String studentName;        // variable
   int totalMarks;            // variable
```

```
    void calculateOverallPercentage() {  // method
        // logic
    }
}
```
Why: Improves readability (totalMarks is easier to read than totalmarks or Totalmarks).

3. Use PascalCase for class names

PascalCase: Every word starts with a capital letter.

```
public class StudentDetails {
    // Class content
}

public class EmployeeInfo {
    // Class content
}
```
Why: It's a Java convention to quickly identify classes by their capitalized names.

4. Use UPPERCASE_SNAKE_CASE for constants

Constants are declared using static final and their names are in all caps with underscores.

Example:

```
public class AppConfig {
    public static final int MAX_USERS = 100;
    public static final String DEFAULT_ROLE = "USER";
}
```
Why: It distinguishes constants from variables and helps other developers know the value shouldn't change.

Combined Example:

```
public class BankAccount {

    private String accountHolderName;       // camelCase
    private double accountBalance;          // camelCase
    public static final double MIN_BALANCE = 1000.00;  // UPPERCASE_SNAKE_CASE

    public void depositAmount(double amount) {  // camelCase
        accountBalance += amount;
    }

    public static void main(String[] args) {
        BankAccount account = new BankAccount();  // PascalCase for class
        account.accountHolderName = "Amit";
        account.accountBalance = 5000;
        account.depositAmount(2000);
    }
}
```

Q3. What are literals?

Answer: Literals are the values used in java in any expression or individually. There are 5 literals in java. They are:

1. Integer Literals
2. Real Literals
3. Character Literals
4. Boolean Literals
5. String Literals

e.g:

```
    int count = 700;        // 700 is Integer literal
double price = 99.99;   // 99.99 is Floating-point literal
char grade = 'J';       // J Character literal
boolean isEven = true; // true is Boolean literal
    String name = “Jagannath”; // Jagannath is String literal
```

Integer Literals:

Any positive or negative integer is considered as integer literal.
0 is also integer literal.
Integer literals can have suffix L/l to represent long values.
e.g: 90,345,968,-123,-7834,-67823 etc.

- Integer Literals can be stored in integer variables
- Variables used to store integer literals are created by the help of primitive data types.
- Primitive data types are used to create variables that can hold integer literals are:
  - **byte**
  - **short**
  - **int**
  - **long**

In Java, integer literals represent whole numbers. They can be expressed in different number systems:

a) Decimal literal (Base 10)
b) Octal literal (Base 8)
c) Hexadecimal literal (Base 16)
d) Binary Literal (Base 2) (Available from JDK 1.7 onwards)

Decimal Integer Literals (Base 10)

- Uses digits 0 to 9.
- It is the default number system in Java.
- No prefix is needed.

```
public class DecimalExample {
   public static void main(String[] args) {
      int num1 = 100;
      int num2 = -250;
      int num3 = 0;   // Zero is also a valid decimal literal

      System.out.println("Decimal Numbers:");
      System.out.println(num1);  // Output: 100
      System.out.println(num2);  // Output: -250
      System.out.println(num3);  // Output: 0
   }
}
```

Octal literal (Base 8)

- Uses digits 0 to 7.

- Prefixed with 0 (zero).
- Digits 8 and 9 are NOT allowed.

```
public class OctalExample {
    public static void main(String[] args) {
        int oct1 = 012;  // 12 (octal) → 10 (decimal)
        int oct2 = 076;  // 76 (octal) → 62 (decimal)
        int oct3 = 00;   // 0 (octal) → 0 (decimal)

        System.out.println("Octal Numbers:");
        System.out.println(oct1);  // Output: 10
        System.out.println(oct2);  // Output: 62
        System.out.println(oct3);  // Output: 0
    }
      }
```

Hexadecimal literal (Base 16)

- Uses digits 0 to 9 and letters A to F (case-insensitive).
- Prefixed with 0x or 0X (zero followed by 'x' or 'X').
- A to F (or a to f) represent 10 to 15 in decimal.

```
public class HexadecimalExample {
    public static void main(String[] args) {
        int hex1 = 0x1A;   // 1A (hex) → 26 (decimal)
        int hex2 = 0XFF;   // FF (hex) → 255 (decimal)
        int hex3 = 0x0;    // 0 (hex) → 0 (decimal)
        int hex4 = 0xabcdef; // Hex representation
        System.out.println("Hexadecimal Numbers:");
        System.out.println(hex1);  // Output: 26
        System.out.println(hex2);  // Output: 255
        System.out.println(hex3);  // Output: 0
        System.out.println(hex4);  // Output: 11259375
    }
      }
```

Binary Literal (Base 2) (Available from JDK 1.7 onwards)

- Uses only 0 and 1.
- Prefixed with 0b or 0B (zero followed by 'b' or 'B').

```
public class BinaryExample {
    public static void main(String[] args) {
        int bin1 = 0b1010;  // 10 in decimal
        int bin2 = 0B1101;  // 13 in decimal
        int bin3 = 0b0001;  // 1 in decimal

        System.out.println("Binary Numbers:");
        System.out.println(bin1);  // Output: 10
        System.out.println(bin2);  // Output: 13
        System.out.println(bin3);  // Output: 1

        //int invalidBin = 0b1201; // ERROR: 2 is not allowed in binary
    }
      }
```

Real Literals: Any fractional/decimal number can be considered as real literal.

e.g: 43.7843, -9046.67, 154.72F, 153.95f, 456.76D, 45.678d etc.
Real literals may end with suffix 'f'/'F' for **float** and 'd'/'D' for **double**.
In Java, real literals (also called floating-point literals) represent numbers with a fractional part (decimal values). These literals are used to store values of type float and double.

```
public class RealLiteralsExample {
    public static void main(String[] args) {
        // Default double literals
        double d1 = 10.5D;
        double d2 = 3.14159;

        // Using 'd' explicitly (optional)
        double d3 = 2.5d;

        // Float literals using 'f' or 'F'
        float f1 = 3.14f;
        float f2 = 0.123F;

        // Scientific notation (exponential form)
        double exp1 = 3.14e2;  // 3.14 * 10^2 = 314.0
        double exp2 = 2.5E-3;  // 2.5 * 10^-3 = 0.0025

        // Printing values
        System.out.println("Double d1: " + d1);
        System.out.println("Double d2: " + d2);
        System.out.println("Double d3: " + d3);
        System.out.println("Float f1: " + f1);
        System.out.println("Float f2: " + f2);
        System.out.println("Exponential exp1: " + exp1);
        System.out.println("Exponential exp2: " + exp2);
    }
}
```

Q1: Why does this code cause an error? How do you fix it?

```
public class Test {
    public static void main(String[] args) {
        float num = 3.14;
        System.out.println(num);
    }
}
```

Answer:
This causes a compilation error because 3.14 is a double by default, and it cannot be assigned to a float without explicit conversion.
Fix: Add f suffix.

Q2: What is the output of the following code?

```
public class Test {
    public static void main(String[] args) {
        double x = 2.5E3;
        float y = 5.0f;
        System.out.println(x + y);
    }
}
```

Answer:
2.5E3 means 2.5 * 10^3 = 2500.0

y = 5.0f (**float**)
Hence, the result will be 2500.0 + 5.0 = **2505.0**

Q3: Predict the output of this code

```
public class Test {
    public static void main(String[] args) {
        double a = 0.1 * 7;
        double b = 0.7;

        System.out.println(a == b);
    }
}
```

Answer:_____

Q4. Can you store a real literal in an int variable? Why or why not?

```
public class Test {
    public static void main(String[] args) {
        int x = 5.5;
        System.out.println(x);
    }
}
```

Answer: Compilation Error: You cannot assign a floating-point number (5.5) to an int directly.
Fix: Use type casting.

```
int x = (int) 5.5;  // x = 5 (decimal part is truncated)
```

Q5: Why does float f = 1/3; not give an expected result?

```
public class Test {
    public static void main(String[] args) {
        float f = 1 / 3;
        System.out.println(f);
    }
}
```

Answer:
1 / 3 performs integer division, which results in 0.

Fix: Convert one operand to **float**.

```
float f = 1.0f / 3;
```

Character Literals: Any single digit, symbol or letter within single quote is considered as character literal.
Symbols: '@', '%', '&',' '....

Boolean Literals: Any conditional value is considered as boolean literal. There are two boolean literals and they are always written in lowercase letter only.
They are : true, false.

String literal: Any literal enclosed within double quotes "" is considered as String literal.

null literal: It is the default value for referenced variables when they are declared as static variable or instance variable.

Chapter 3. Data Types and Types of Data Types

Q. What do you mean by data type?
Answer: A data type is a keyword or word which is used to create variable, and variable will store single or multiple value at a time.

Q. What is the syntax to create a variable?
Answer:
<datatype> <variablename>;

Q. Write a snippet to create a variable which will be used to store age of an employee working in NareshiTechnologies.
Answer:

```
byte empAge = 57;
or
int empAge = 60;
```

Q. What are the different types of data types?
Answer: There are two different types of data types:
- Primitive Data Types
- Referenced Data Types

Q. What do you mean by primitive data types?
Answer: The data types which are used to create variables that can store only one value at a time are called as primitive data types.
There are 8 different Primitive Data Types:

Primitive Data Type (PDT)	Memory Size	Wrapper Class	Range / Values
byte	1 byte	Byte	-128 to 127
short	2 bytes	Short	-32,768 to 32,767
int	4 bytes	Integer	-2,147,483,648 to 2,147,483,647
long	8 bytes	Long	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Float	Approx. ±3.4e38 (6-7 decimal digits)
double	8 bytes	Double	Approx. ±1.8e308 (15 decimal digits)
char	2 bytes	Character	Unicode characters: '\u0000' (0) to '\uffff' (65,535)
boolean	~1 bit (JVM-dependent)	Boolean	true or false

```
class TestByte{
public static void main(String []args){
 byte num;    //variable 'num' is declared/created
 num = 100;   //variable 'num' has been initialized with 100
 System.out.println("Value in variable is: "+num);
}
}
------
```

Output:
Value in variable is: 100

```
class TestShortRange{
public static void main(String []args){
    short sh;//here variable sh will consume 2bytes of memory
  System.out.println("Max value for a short type variable: "+Short.MAX_VALUE);
  System.out.println("Min . value for a short type variable: "+Short.MIN_VALUE);
}
}
```

Output:
Max value for a short type variable: -32768
Min . value for a short type variable: 32767

```
class TestIntRange{
public static void main(String []args){
System.out.println("Min. value allowed to be stored in int type variable: "
                    +Integer.MIN_VALUE);
System.out.println("Max. value allowed to be stored in int type variable: "
                    +Integer.MAX_VALUE);
}
}
```

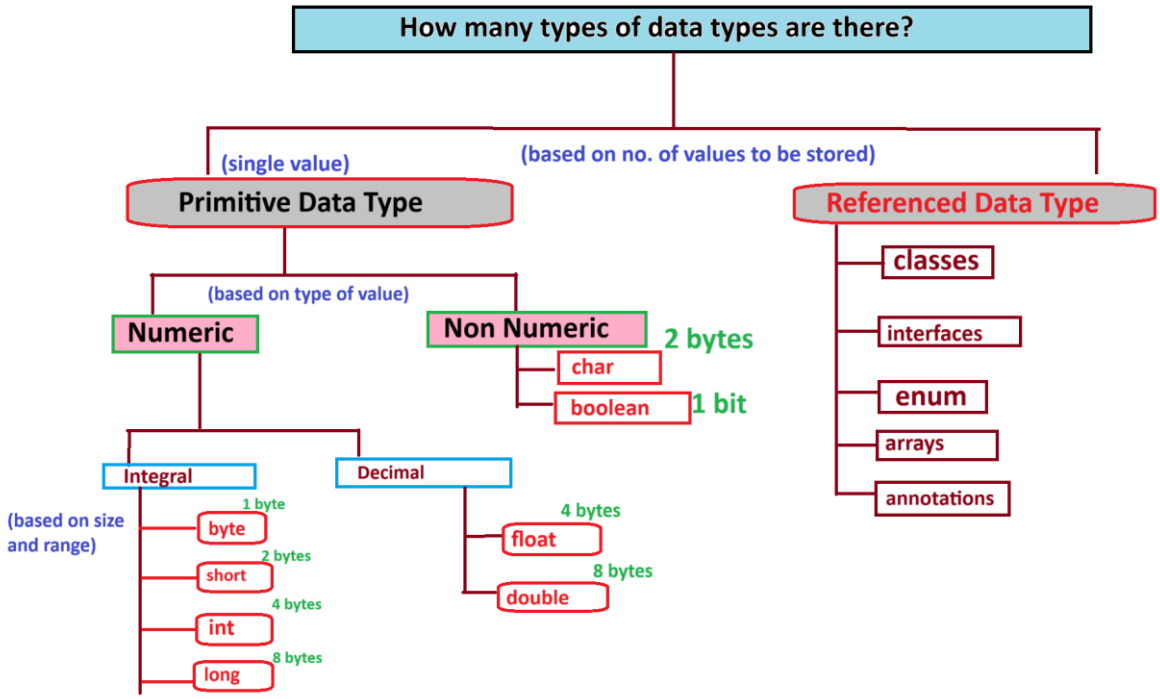
Output:
Min. value allowed to be stored in int type variable: -2147483648
Max. value allowed to be stored in int type variable: 2147483647

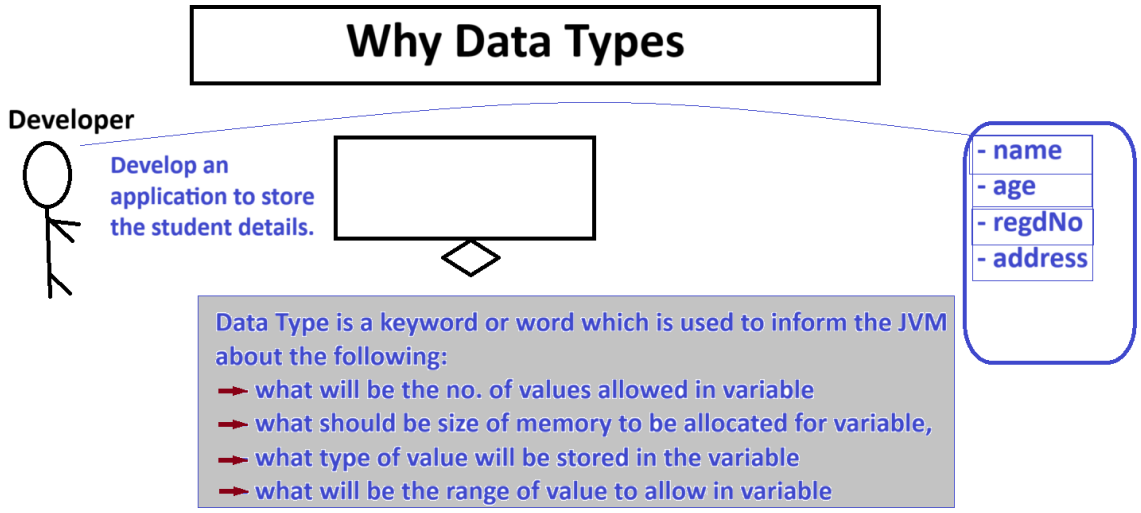
```
class TestLongRange{
public static void main(String []args){
System.out.println(Long.MIN_VALUE);
System.out.println(Long.MAX_VALUE);

}
}

 class TestByte{
public static void main(String []args){
 byte num;    //variable 'num' is declared/created
 num = -127;   //variable 'num' has been initialized with 100
 System.out.println("Value in variable is: "+num);
}
}
```

Q1. Write a program to create two variables, where you can store two different integers. Display the result for all forms of relational operations on the two values stored in the variables.
You must provide readable mesaage.
Solution:

```
class TestRelation{
public static void main(String []args){
   int a,b;
   a = 10;
   b = 20;
   System.out.println(a+" < "+b+" : "+(a<b));
}
```

```
}
-----------------------------------------------------------------
//Q. WAP to store three integers. Calculate and display their sum.
class Calculator{
public static void main(String []args){
 int a,b,c,sum;//declaration
 a=10;//initialization
 b=20;//initialization
 c=10;//initialization
 sum = a+b+c;
 System.out.println("Sum of "+a+" , "+b+" and "+c+" : "+sum);
}
}
```





Referenced Data Types:

The data types which are used to create referenced variables are called as referenced data types.

Referenced variables are used to store the reference of an object.

We store the reference of an object into a reference variable to access the non static variables and non static methods loaded inside the object using the reference variable.

Types of referenced data types:

- classes
- interfaces
- array
- annotation
- enum

Q. What is a class?

Answer: A class is a blueprint of objects which means by looking at the class name one can get an idea regarding the category of object to be utilized.

Note: A class is also termed as 'object factory' beacuse it is used to create objects.

Q. How do we create a class?

Answer: We create a class by the help of keyword 'class'.

Q. What is the syntax of creating a class?

Answer:

<class> <class name>{

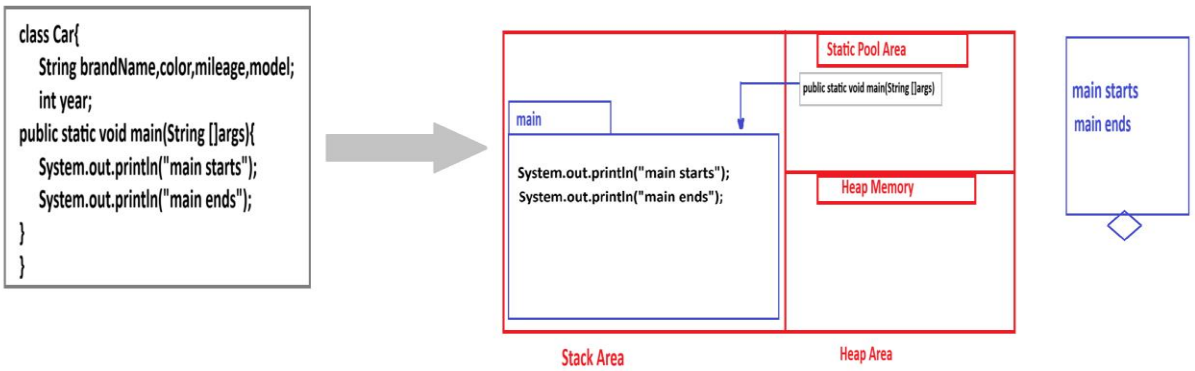
}
e.g:
class Employee{

}

Here we created a class as Employee which means it will be used to represent real world employees and will have functionalities related to employees.

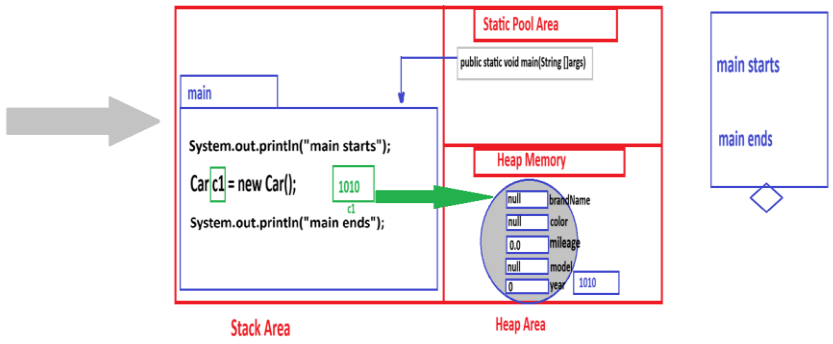
//WAP to represent Cars of your choice.
Note:cars will have different brandName,color,mileage,model,year.
Create variables to represent the mentioned attriutes.
Store values for a car created and display them with readable message.
(Only one car created).Make execution process diagram.

Car Program without objects doesnot load the nonstatic variables into the memory.



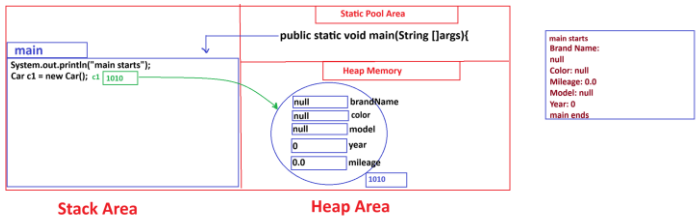
Car Program with car object created lets the non static variable get memory allocated to them, but with default values depending on the data type used.

```
class Car{
    String brandName,color,        ,model;
    int year; double,mileage;
public static void main(String []args){
    System.out.println("main starts");
    Car c1 = new Car();
    System.out.println("main ends");
}
}
```



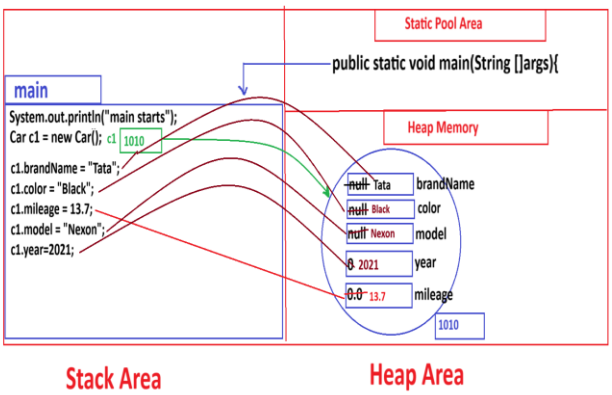
Displaying non static variables without initializing them by the help of reference variables will give default values:

```
class Car{
    String brandName,color,model;
    int year;
    double mileage;
public static void main(String []args){
    System.out.println("main starts");
    //To get the memory allocation for non static variables we must create objects
    Car c1 = new Car();
    System.out.println("Brand Name: "+c1.brandName);
    System.out.println("Color: "+c1.color);
    System.out.println("Mileage: "+c1.mileage);
    System.out.println("Model: "+c1.model);
    System.out.println("Year: "+c1.year);
    System.out.println("main ends");
}
}
```



Instance variables(Non-static variables) are initilaized by the help of object reference nd displayed.

```
class Car{
    String brandName,color,model;
    int year;
    double mileage;
public static void main(String []args){
    System.out.println("main starts");
    //To get the memory allocation for non static variables we must create objects
    Car c1 = new Car();
    c1.brandName="Tata";
    c1.color = "Black";
    c1.mileage = 13.7;
    c1.model = "Nexon";
    c1.year=2021;

}
}
```



======= Chapter 4. Operators ==========

We know how to store values using java in variables which are created using data types. We have stored single value as well as multiple values.
From storing the value, to perform operations ,such as validate the data based on some condition or perform calculations, we need operators to perform these types of operations.

Q.What is an operator?
Answer: An operator is a symbol or a word which is used to perform operations such as calculations, create new object, assignment and validations for any operand or operands.

(NOTE: 'new' is also an operator
'instanceof' is also an operator).

There are **40 operators.**

Types of operators based on number of operands used:
i. Unary- used with single operand. ++,--, +,-,~,!,new Class().
ii. Binary - used with minimum two operands.
iii.Ternary-Its an alternative to if...else.
(expression)?val1:val2

In detailed list of 40 operators:

1. **Assignment -** for assigning values '='
= is used to assign a value to a variable.
For e.g:
int x = 100;
100 is assigned to variable 'x' by using =

char ch = 'J';
J is assigned to variable ch by using =.
- We can use assignment operator to store value into a variable.

- We can store a result of expression into a variable, we can also store value returned from a method into variable.
e.g:
int stringLength = "java".length();
Here length() returns the number of characters in 'java' i.e, 4.
4 is stored in stringLength as the value returned from method is now assigned to the variable.

What is the difference between '=' and '=='?
Answer:
i) = is termed as assignment operator whereas == is a relational operator.
ii) = is used to assign value to a variable whereas == is used to compare the equality of primitive type values only.

Note: == can be used with primitive values to check the equality of values.
== can be used with the referenced variables to check the equality of reference of the objects.
equals() is used to check the equality of content of reference variables.

- Anything within "" is considered as a string object in java.
- We can create string objects in two ways:
i). directly
ii). using new keyword

//creating string object directly

```
class TestString{
public static void main(String []args){
  String s1 = "hello";//created string object directly
  String s2 = "java";//created string object directly
  String s3 = "hello";//created string object directly
}
}
```

******NOTE:**
- Whenever a string object is created directly, it is always loaded into SCP Area(String Constant Pool Area).
- JVM always looks for availability of object in the SCP area with the same content that we are creating,if any object is available with the same content then, no new object is created in the SCP Area rather old object reference is shared with the reference variable of the object that we are creating.

//creating string object new keyword

```
class TestString{
public static void main(String []args){
 String s1 = "java";//created string object directly
 String s2 = new String("java");//created string object using 'new' keyword
}
}
```

Note: Whenever we create string object using 'new' keyword then the object is loaded into Heap Memory. In this case no checking of availability of object with same content is performed.

***/**

```
class TestEquality{
public static void main(String []args){
/*

int num1 = 100;
int num2 = 100;
System.out.println(num1 == num2);//true

byte b1 = 120;
int b2 = 120;
System.out.println(b1 == b2);//true

char ch1 = 'A';
char ch2 = 'a';
System.out.println(ch1 == ch2);//false
*/
```

```
//Note: equals() is used to check the equality of object content.
String s1 = "java";
String s2 = new String("java");
//System.out.println(s1 == s2); reference comparison
System.out.println(s1.equals(s2));//content comparison
}
}
/*equals() cannot be used with primitive types, it can be used only with
  referenced types */

/* Program to demonstrate the arithmetic operators and their use.
+ --> used to add values
- --> used to subtract values
* --> used to multiply values
/ --> used to divide values and fetch the quotient
% --> used to divide values and fetch the remainder
*/
class TestArithmeticOperator
{
        public static void main(String []args){
        int a = 10, b = 5;
        System.out.println("Sum of a and b: "+(a+b));
        System.out.println("Difference of a and b: "+(a-b));
        System.out.println("Product of a and b: "+(a*b));
        System.out.println("Quotient for a / b: "+(a/b));
        System.out.println("Reaminder for a / b: "+(a%b));
        }
}
```

2. Relational - for validations <, >, <=, >=, ==, != , instanceof
3. Arithmetic - for calculations + , - , *, / , %
4. Logical - &&, ||, !
5. Increement- ++
6. Decreement- --
7. Ternary- ? :
8. Shift- <<, >>, >>>
9. Compound Assignment- += ,-=, *=, /=, %=, &=, |=, ^=, <<=, >>=, >>>=
10. Bitwise- &, |, ^, ~
11. Lambda- -> (Introduced in Java 8)
12. Object Creation- new
(datatype) cast is also considered as operator.

Q. What is an operand?
Answer: A value which is used in expression on which operation is performed is called as operand.We can use literal,variable,expression, non void method call, or object as operands.

Q.What is an expression?
Answer:Combination of operands and operators together to perform any operation is called as Expression.Types of expressions:

i). Pure Expression: same type of operands used
ii). Mixed Expression: different types of operands used

Q. What is operator precedence?
Answer: The order of execution of operators in an expression is called operator precedence. If we have multiple combination of operators then which operator will execute first is decided by the operator precedence.

Q. What is operator associativity?
Answer: The direction of execution of operator from left to right or right to left is called operator associativity.
Only 2 operators execute Right to Left i.e.:
'=' and 'cast operator'.
Remaining all operators execute Left to Right.

Diagram of Operator precedence: Check the pic attached with this material. or check the link:

https://drive.google.com/drive/folders/1QQnJjfdI_wsTzYvWNgKz2Kl4bMITeFgA?usp=sharing

---

Note:
In an expression, unary operators execute first.
postfix ++,postfix --, prefix++,prefix--, ~,!,new

In an expression containing arithmetic operations, first multiplicative operators execute
*, /, %
then additive operators execute: +, -

Operators in same row execute in the order they are available in expression.

Arithmetic Operators:
-The operators which are used to perform calculations such as addition, subtraction,multiplication and division are called arithmetic operators.
-The arithmetic operators are binary type operators.
-We have 5 arithmetic operators: +,-,*,/,%
-We are not allowed to use A.O with boolean type values.
-We are allowed to use AO with byte,short,int,long,float,double,char values
-Arithmetic Operator '+' concats two S.O.

Points to remember:

---

1. **byte + byte = int**
2. **short + short = int**
3. **char + char = int**

---

4. **int + int = int**
5. **long + long = long**
6. **float + float = float**
7. **double + double = double**

- If we use literals in expression then compiler uses value and generates result and then verifies whether the result is in range of destination variable or not. If yes then allowed, else not allowed to store and throws possible lossy conversion.

- If we use variable in expression then compiler uses variable

type and generates type as value and then verifies whether the type range of result is in range of type used for result or not.

- In expression execution,it always generates numeric result on the basis of data type used in expression.The result is always the highest range data type used in expression.

-Hierarchy for type of data type (Low to High)

byte-short - int-long-float-double-
char

* char is less than int.

byte<short<int<long<float<double

Relational operators:

In Java relational operators are used to compare two values.
They return a boolean result (true or false). These operators are commonly used in conditional statements, loops, and expressions where decision-making is required.

```
public class RelationalOperatorsExample {
    public static void main(String[] args) {
        int a = 10, b = 20;
        System.out.println("a == b : " + (a == b));  // false
        System.out.println("a != b : " + (a != b));  // true
        System.out.println("a > b  : " + (a > b));   // false
        System.out.println("a < b  : " + (a < b));   // true
        System.out.println("a >= b : " + (a >= b));  // false
        System.out.println("a <= b : " + (a <= b));  // true
    }
}
```

Logical Operators

Logical operators in Java are used to perform logical operations on boolean expressions. These operators return a boolean value (true or false) based on the logical relationship between operands.

```
public class LogicalOperatorsExample {
public static void main(String[] args) {
int a = 15, b = 5, c = 20;

// Logical AND (&&)
System.out.println("Logical AND (&&):");
System.out.println((a > 10 && b < 10));
 System.out.println((a < 10 && c > 15));

// Logical OR (||)
System.out.println("\nLogical OR (||):");
System.out.println((a > 10 || b > 10));
System.out.println((a < 10 || c < 15));

// Logical NOT (!)
System.out.println("\nLogical NOT (!):");
System.out.println(!(a > 10)); // !true → false
System.out.println(!(b > 10)); // !false → true
}
```

```
}
```

Increment and Decrement Operators in Java

In Java, the increment (++) and decrement (--) operators are used to increase or decrease the value of a variable by 1. They can be used in two forms:
Postfix (x++ or x--) – Value is used first, then updated.
Prefix (++x or --x) – Value is updated first, then used.

1. Increment Operator (++)

Post-Increment (x++): Returns the current value of x, then increments x by 1.
Pre-Increment (++x): Increments x by 1, then returns the new value.

e.g:

```
public class IncrementExample {
    public static void main(String[] args) {
        int a = 5;
        int b = a++; // Post-increment
        System.out.println("a: " + a);
        System.out.println("b: " + b);

        int c = 5;
        int d = ++c; // Pre-increment
        System.out.println("c: " + c);
        System.out.println("d: " + d);
    }
}
```

2. Decrement Operator (--)

Post-Decrement (x--): Returns the current value of x, then decrements x by 1.
Pre-Decrement (--x): Decrements x by 1, then returns the new value.
e.g:

```
public class DecrementExample {
    public static void main(String[] args) {
        int x = 5;
        int y = x--; // Post-decrement
        System.out.println("x: " + x);
        System.out.println("y: " + y);

        int p = 5;
        int q = --p; // Pre-decrement
        System.out.println("p: " + p);
        System.out.println("q: " + q);
    }
}
```


Ternary operator (? :)

Ternary Operator in Java is a shorthand for an if-else statement.
It is also called the conditional operator because it evaluates a condition and returns one of two values based on whether the condition is true or false.

Syntax:

```
variable = (condition) ? expression1 : expression2;
```

```java
public class TernaryExample {
    public static void main(String[] args) {
        int a = 10, b = 20;
        int min = (a < b) ? a : b; // Finds the smaller value
        System.out.println("Minimum value: " + min);
    }
}
```

Ternary operator replacing if else:

```java
if (age >= 18) {
    status = "Adult";
} else {
    status = "Minor";
}
```

```java
String status = (age >= 18) ? "Adult" : "Minor";
```

Nested Ternary:

```java
public class NestedTernaryExample {
    public static void main(String[] args) {
        int num = 0;
        String result = (num > 0) ? "Positive" : (num < 0) ? "Negative" : "Zero";
        System.out.println("The number is: " + result);
    }
}
```

Ternary with method call:

```java
public class TernaryMethodExample {
    public static void main(String[] args) {
        int x = 7, y = 10;
        System.out.println("Maximum: " + getMax(x, y));
    }

    public static int getMax(int a, int b) {
        return (a > b) ? a : b;
    }
}
```

Disadvantages of Ternary Operator:

- **Less readable when used excessively or nested too deeply**
- **Cannot execute multiple statements (only one per branch)**
- **Not suitable for complex logic (use if-else instead)**

Shift Operators in Java

Shift operators in Java are used to manipulate bits by shifting them left or right. These operators work on integer types (byte, short, int, long) and help in optimizing performance, especially in bitwise operations.

Types of Shift Operators

1. Left Shift (<<)
2. Right Shift (>>)
3. Unsigned Right Shift (>>>)

1. Left Shift Operator (<<)

The left shift (<<) operator moves bits to the left by the specified number of positions.
Each left shift multiplies the number by 2.

Syntax
result = number << shiftCount;

e.g:

```
public class LeftShiftExample {
    public static void main(String[] args) {
    int a = 5; // Binary: 0000 0101
    int result = a << 2; // Moves bits 2 places left (5 * 2^2 = 20)

    System.out.println("5 << 2 = " + result); // Output: 20
    }
}
```

Explanation:
5 (0000 0101) becomes 20 (0001 0100) after shifting 2 bits to the left.
Formula: number * (2^shiftCount)

2. Right Shift Operator (>>)

The right shift (>>) operator moves bits to the right.
Each right shift divides the number by 2 and keeps the sign (preserves the sign bit).

Syntax
result = number >> shiftCount;

e.g:

```
public class RightShiftExample {
    public static void main(String[] args) {
    int a = 20;  // Binary: 0001 0100
    int result = a >> 2; // Moves bits 2 places right (20 / 2^2 = 5)

    System.out.println("20 >> 2 = " + result); // Output: 5
    }
}
```

Explanation:
20 (0001 0100) becomes 5 (0000 0101) after shifting 2 bits to the right.
Formula: number / (2^shiftCount)

3. Unsigned Right Shift (>>>)

The unsigned right shift (>>>) moves bits to the right but fills empty spaces with 0.
It ignores the sign bit (always fills with 0), making negative numbers positive.

Syntax

result = number >>> shiftCount;

e.g:

```
public class UnsignedRightShiftExample {
public static void main(String[] args) {
int a = -20;// Binary (32-bit): 1111 1111 1111 1111 1111 1111 1110 1100
int result = a >>> 2; // Moves bits 2 places right (fills with 0)
System.out.println("-20 >>> 2 = " + result);
   }
}
```

Explanation:

- Unlike >>, which keeps the sign, >>> fills empty places with 0, making a large positive number.

Operator	Symbol	Effect
Left Shift	<<	Shifts bits to the **left** (multiplies by 2^n)
Right Shift	>>	Shifts bits to the **right**, keeping the sign (divides by 2^n)
Unsigned Right Shift	>>>	Shifts bits to the **right**, filling with 0 (divides by 2^n)

Compound Assignment Operators

In Java, compound assignment operators are shorthand notations that combine an arithmetic or bitwise operation with assignment. These operators help in writing cleaner and more concise code.

Syntax:
variable op= expression;
where op is an arithmetic or bitwise operator.

Operator	Equivalent Expression	Description
+=	a = a + b;	Addition assignment
-=	a = a - b;	Subtraction assignment
*=	a = a * b;	Multiplication assignment
/=	a = a / b;	Division assignment
%=	a = a % b;	Modulus assignment
&=	a = a & b;	Bitwise AND assignment
^=	a = a ^ b;	Bitwise XOR assignment
<<=	a = a << b;	Left shift assignment
>>=	a = a >> b;	Right shift assignment
>>>=	a = a >>> b;	Unsigned right shift assignment

```
public class CompoundAssignmentExample {
   public static void main(String[] args) {
      int a = 10;
      int b = 5;

      a += b;  // a = a + b => 10 + 5 = 15
      System.out.println("After += : " + a);

      a -= b;  // a = a - b => 15 - 5 = 10
      System.out.println("After -= : " + a);
```

```
        a *= b;  // a = a * b => 10 * 5 = 50
        System.out.println("After *= : " + a);

        a /= b;  // a = a / b => 50 / 5 = 10
        System.out.println("After /= : " + a);

        a %= b;  // a = a % b => 10 % 5 = 0
        System.out.println("After %= : " + a);

        a = 10;
        a &= b;  // a = a & b => 10 & 5 = 0 (bitwise AND)
        System.out.println("After &= : " + a);

        a = 10;
        a |= b;  // a = a | b => 10 | 5 = 15 (bitwise OR)
        System.out.println("After |= : " + a);

        a = 10;
        a ^= b;  // a = a ^ b => 10 ^ 5 = 15 (bitwise XOR)
        System.out.println("After ^= : " + a);

        a = 10;
        a <<= 1; // a = a << 1 => 10 << 1 = 20 (left shift)
        System.out.println("After <<= : " + a);

        a = 10;
        a >>= 1; // a = a >> 1 => 10 >> 1 = 5 (right shift)
        System.out.println("After >>= : " + a);

        a = -10;
        a >>>= 1; // a = a >>> 1 (unsigned right shift)
        System.out.println("After >>>= : " + a);
    }
}
```

'new' Operator in Java

The new operator in Java is used to create objects (instances) of a class. When you use new, memory is allocated in the heap for the object, and the constructor of the class is called to initialize it.

Static Example

```
class MathUtility {
    static int square(int x) {
        return x * x;
    }
}

public class Demo {
    public static void main(String[] args) {
        System.out.println(MathUtility.square(5));  // Output: 25
    }
}
```

- **Here, no object of MathUtility is created.**
- **Static method square() is directly accessed using class name.**

Non Static Example:

```
class Person {
    String name;

    void displayName() {
        System.out.println("Name: " + name);
    }
}

public class Test {
    public static void main(String[] args) {
        Person p = new Person();  // Object creation using 'new'
        p.name = "Jagannath";
        p.displayName();  // Output: Name: Jagannath
    }
}
```

- Here, displayName() and name are non-static.
- So, an object p is created using new Person() to access them.

Summary

- new operator is used to create objects in Java.
- Static members do **not** need an object to access.
- Non-static (instance) members **require** an object.
- Use ClassName.memberName for static members.
- Use objectRef.memberName for non-static members.

Q. Can we access a non-static method from a static method directly?
Answer: No, we need to create an object.

Q. Is memory allocated for static members when object is created?
Answer: No, static members get memory only once at class loading time.

Practise:
Q1.

```
 byte b1 = 10;
 byte b2 = 20;
 System.out.println(b1+b2);
```

Q2.

```
 byte b1 = 10;
 byte b2 = 20;
 byte result = b1 + b2;
 System.out.println(b1+b2);
```

Q3.

```
byte b1 = 10;
byte b2 = 20;
byte result = (byte)b1 + b2;
System.out.println(result);
```

Q4.

```
byte b1 = 10;
byte b2 = 20;
byte result = (byte)b1 + (byte)b2;
System.out.println(result);
```

Q6.

```
byte b1 = 10;
byte b2 = 20;
byte result = (byte)(b1 + b2);
System.out.println(result);
```

Q7.

```
byte b1 = 10;
byte b2 = 20;
byte result = 10+20;
System.out.println(result);
```

Q8.

```
byte b1 = 10;
byte b2 = 20;
byte result = 110+120;
System.out.println(result);
```

Q9.

```
byte b1 = 10;
byte b2 = 20;
byte result = 10+20;
System.out.println(result);
```

Q10.

```
byte b1 = 10L;
byte b2 = 20;
byte result = b1+b2;
System.out.println(result);
```

Q11.

```
byte b1 = (byte)10L;
byte b2 = 20;
byte result = b1+b2;
System.out.println(result);
```

Q12.

```
byte b1 = (int)10L;
byte b2 = 20;
byte result = b1+b2;
System.out.println(result);
```

Q13.

```
long b1 = 10L;
byte b2 = b1;
byte result = b1+b2;
System.out.println(result);
```

Q14.

```
long b1 = 10L;
byte b2 = (byte)b1;
byte result = b1+b2;
System.out.println(result);
```

Q15.

```
long b1 = 10L;
byte b2 = (int)b1;
byte result = b1+b2;
System.out.println(result);
```

Q16.

```
byte b1 = 10;
byte b2 = 20;
byte result = b1 + 20;
System.out.println(result);
```

Q17.

```
final byte b1 = 10;
byte b2 = b1+10;
byte result = b2;
System.out.println(result);
```

Q18.

Q19.

```
final byte b1;
final byte b2;
b1= 10;
b2= 20;
byte result = b1 + b2;
System.out.println(result);
```

Q20.

```
int i1 = 10;
int i2 = 20;
int result = i1+i2;
System.out.println(result);
```

Q21.

```
int i1 = 10;
String i2 = "20";
int result = i1+i2;
System.out.println(result);
```

Q22.

```
int i1 = 10;
```

```
String i2 = "20";
String result = i1+i2;
System.out.println(result);
```

Q23.

```
String i1 = "10"+"JB";
String i2 = "20";
String result = i1+i2;
System.out.println(result);
```

Q24.

```
int i1 = "10";
String i2 = "20";
String result = i1+i2;
System.out.println(result);
```

Q25.

```
int i1 = "10";
String i2 = "20";
String result = i1-i2;
System.out.println(result);
```

Q26.

```
String i1 = "10";
String i2 = "20";
String result = i1*i2;
System.out.println(result);
```

Q27.

```
String i1 = "10";
String i2 = "20";
String result = i1/i2;
System.out.println(result);
```

Q28.

```
String i1 = "10";
String i2 = "20";
String result = i1%i2;
System.out.println(result);
```

Q29.

```
char i1 = "10";
String i2 = "20";
String result = i1+i2;
System.out.println(result);
```

Q30.

```
char i1 = "10";
String i2 = "20";
String result = i1+i2;
System.out.println(result);
```

Q31.

```
char i1 = '10';
String i2 = "20";
```

```
String result = i1+i2;
System.out.println(result);
```

Q32.
```
char i1 = 'A';
String i2 = "20";
String result = i1+i2;
System.out.println(result);
```

Q33.
```
char i1 = 'A';
char i2 = "A";
String result = i1+i2;
System.out.println(result);
```

Q34.
```
char i1 = 'J';
char i2 = 'B';
char result = i1+i2;
System.out.println(result);
```

Q35.
```
char i1 = 'J';
char i2 = 'B';
String result = i1+i2;
System.out.println(result);
```

Q36.
```
char i1 = (char)"J";
char i2 = 'B';
char result = i1+i2;
System.out.println(result);
```

Q37.
```
char i1 = (char)"J";
char i2 = (char)"B";
char result = i1+i2;
System.out.println(result);
```

Q38.
```
char i1 = (char)'J';
char i2 = (char)'B';
char result = i1+i2;
System.out.println(result);
```

Q39.
```
char i1 = (char)((int)'J');
char i2 = (int)'B';
char result = i1+i2;
System.out.println(result);
```

Q40.
```
char i1 = (int)'J';
char i2 = (int)'B';
int result = i1+i2;
```

```
System.out.println(result);
```

Q41.
```
char i1 = (int)'J';
char i2 = (int)'B';
char result = (int)(i1+i2);
System.out.println(result);
```

Q42.
```
char i1 = ()'J';
char i2 = ()'B';
char result = i1+i2;
System.out.println(result);
```

Q43.
```
char i1 = ()'J';
char i2 = ()'B';
int result = (char)i1+i2;
System.out.println(result);
```

Q44.
```
char i1 = 65;
char i2 = 65;
char result = i1+i2;
System.out.println(result);
```

Q45.
```
char i1 = 65;
char i2 = 65;
char result = i1+i2;
System.out.println(result);
```

Q46.
```
char i1 = 'a';
char i2 = 'b';
char result = (char)(i1+i2);
System.out.println(result);
```

Q47.
```
char i1 = 65;
char i2 = 65;
char result = i1+i2;
System.out.println((int)result);
```

Q48.
```
char i1 = 65;
char i2 = 65;
char result = i1+i2;
System.out.println(result);
```

Q49.
```
String i1 = "";
char i2 = 65;
System.out.println(i1+i2);
```

Q50.

```
String i1 = "";
String i2 = ""+10+20+'A';
System.out.println(i1+i2);
```

Q51.

```
String s1 = 10+20;
String s2 = "";
String result = s1+s2;
System.out.println(result);
```

Q52.

```
String s1 = 10+20+"";
String s2 = "";
String result = s1+s2;
System.out.println(result);
```

Q53.

```
String s1 = "";
String s2 = "";
String result = s1+s2;
System.out.println(result);
```

Q54.

```
int s1 = "";
String s2 = "A";
String result = s1+s2;
System.out.println(result);
```

Q55.

```
int s1 = 10;
int s2 = 20;
String s3 = s1+s2;
System.out.println(result);
```

Q56.

```
int s1 = 10;
int s2 = 20;
String s3 = "s1"+s2;
System.out.println(s3);
```

Q57.

```
int s1 = 10;
int s2 = 20;
String s3 = "s1"+"s2";
System.out.println(s3);
```

Q58.

```
int s1 = 10;
int s2 = 20;
String s3 = "s1+s2";
System.out.println(s3);
```

Q59.

```
int s1 = 10;
```

```
int s2 = 20;
String s3 = s1+s2+"s1+s2";
System.out.println(s3);
```

Q60.

```
int s1 = 10;
int s2 = 20;
String s3 = "s1+s2"+s1+s2;
System.out.println(s3);
```

Q61.

```
int s1 = 10;
int s2 = 20;
String s3 = "s1+s2"+(s1+s2);
System.out.println(s3);
```

Q62.

```
int s1 = 10;
int s2 = 20;
String s3 = "s1+s2"+(s1-s2);
System.out.println(s3);
```

Q63.

```
int s1 = 10;
int s2 = 20;
String s3 = "s1-s2"+(s1-s2);
System.out.println(s3);
```

Q64.

```
int s1 = 10;
int s2 = 20;
String s3 = "s1-s2"+(s1-s2);
System.out.println(s3);
```

Q65.

```
int s1 = 10;
int s2 = 20;
System.out.println(22/7*s1*s2);
```

Q66.

```
int s1 = 22;
int s2 = 7;
int s3 = 10;
int s4 = 10;
System.out.println(s1/s2*s1*s2);
```

Q67.

```
int s1 = 0;
int s2 = 7;
System.out.println(s1/s2);
```

Q68.

```
int s1 = 0;
int s2 = 1;
System.out.println(s1/s2);
```

Q69.

```
int s1 = 0;
int s2 = 1;
System.out.println(s2/s1);
```

Q70.

```
int s1 = 2;
int s2 = 1;
System.out.println(s1/s2);
```

Q71.

```
int s1 = 0;
int s2 = 1;
System.out.println(s2/s1);
```

Q72.

```
int s1 = 0;
int s2 = 0;
System.out.println(s1/s2);
```

Q73.

```
int s1 = 2;
int s2 = 1;
System.out.println(s2/s1);
```

Q74.

```
double s1 = 0.0;
int s2 = 1;
System.out.println(s1/s2);
```

Q75.

```
double s1 = 0.0;
double s2 = 1.0;
System.out.println(s2/s1);
```

Q76.

```
double s1 = 0.0;
int s2 = 1;
System.out.println(s2/s1);
```

Q77.

```
double s1 = 1.0;
int s2 = 0;
System.out.println(s1/s2);
```

Q78.

```
double s1 = -1.0;
int s2 = 0;
System.out.println(s2/s1);
```

Q79.

```
int s1 = 0;
int s2 = 0;
System.out.println(s1/s2);
```

Q80.

```
int s1 = 0;
int s2 = 0;
System.out.println(s1/s2);
```


Primitive Data Types:

	Size	Default Value	Example	Description
byte	1 byte	0	byte a = 10;	Small integer (-128 to 127)
short	2 bytes	0	short s = 1000;	Medium integer (-32K to 32K)
int	4 bytes	0	int x = 10000;	Default for integer values
long	8 bytes	0L	long l = 1000000000L;	Larger integers
float	4 bytes	0.0f	float f = 5.75f;	Single-precision decimal numbers
double	8 bytes	0.0d	double d = 19.99;	Double-precision decimals
char	2 bytes	'\u0000'	char c = 'A';	A single character
boolean	~1 bit	false	boolean b = true;	true or false value

Referenced Data Type

Type	Example	Description
String	String s = "Java";	Sequence of characters (class)
Arrays	int[] arr = {1,2};	Collection of similar data types
Classes	Car car = new Car();	User-defined types

Type	Example	Description
Interfaces	Runnable r = ...;	Blueprint for classes
Enums	enum Day { MON, TUE }	Set of constant values

Tips to Remember

- Use primitive types when performance is important.
- Use non-primitive types when you need more functionality (methods, behaviors).
- All non-primitive types store references (addresses), not actual data.
- The default value for object types (non-primitive) is null.

e.g:

```
public class DataTypeExample {
    public static void main(String[] args) {
        int age = 25;
        float weight = 75.5f;
        char grade = 'A';
        boolean isPassed = true;
        String name = "John";

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Weight: " + weight);
        System.out.println("Grade: " + grade);
        System.out.println("Passed: " + isPassed);
    }
}
```

What is OOP?

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects", which contain both **data (fields/variables)** and **behavior (methods)**. OOP helps organize code in a reusable, modular, and maintainable way.

Key Goals of OOP

- Code Reusability
- Modularity
- Scalability
- Security
- Ease of Maintenance

Principles Object-Oriented Programming (OOP) with Real-World Application

Concept	Definition	Real-World Application Example
1. Encapsulation	Wrapping data (variables) and methods (functions) into a single unit (class). It protects the internal state of an object by restricting direct access.	In a **Banking App**, your account balance is private. You access it using secure methods like getBalance(), but can't directly change it.
2. Abstraction	Hiding internal implementation details and showing only essential features to the user.	In **Google Maps**, you search and navigate using a simple UI, while the complex GPS algorithms and API integrations run in the background.

Concept	Definition	Real-World Application Example
3. Inheritance	A mechanism where one class (child) inherits properties and behaviors from another class (parent).	In **e-commerce apps** like Amazon, different product types (Mobile, Laptop, Clothing) inherit from a base Product class with shared features.
4. Polymorphism	The ability of an object to take many forms. A single method behaves differently based on the object calling it.	In **payment gateways** like Razorpay or Paytm, a processPayment() method behaves differently for **UPI**, **credit card**, or **net banking** objects.

Advantages of OOP

- Modular Structure
- Code Reusability
- Easy to debug and maintain
- Flexible and scalable
- Better security through encapsulation

=== Variables and Types of Variables===

- Variable is used to store single or multiple values at a time
- Variable makes the code more readable
- Using variable we can reuse the value
- using variable it is easy for a developer to maintain the program

Q1. What is a variable?
Answer: Variable is a named memory location which is used to store single or multiple values at a time.

Q2. How do we create a variable?
Answer: By using data types.

Q3. How do we declare a variable?
Answer: Syntax:
<datatype> <variablename>;
e.g:

```
int age;  //here age is a variable declared by the help of data type int
```

Q4. Why do we need a variable?
Answer: We need variable to store a value so that we can reuse the value.

Q5. Where do we declare a variable?
Answer: We can declare variables in class,interface,enum,method,block and constructor.
e.g:

```
//Variable inside class
class Sample{
int x;//instance variable
static char ch; //static variable
```

```
}

//variable inside interface: in an interface by default variable is static
interface Operabale{
boolean isAddable=true; //static variable
}

//variable inside method
class Employee{

public static void workHours(double duration){
                        //duration is local variable
double increement;  //local variable

}
}
```

```
//variable inside a block
class Car{

static{
  double mileage = 15; //mileage is a local variable
}

{
  String brand = "Maruti";// brand is a local variable
}
}

//variable inside constructor
class Bank{
   // Bank() is a constructor
  Bank(){
  int deduction = 100; //deduction is local variable
  }

}
```

Q6. What are the types of variables?

Answer: There are 3 types of variables depending upon position of declaration:

- **Static Variable**: Variable declared outside any method,block or constructor by using the keyword static is called as static variable.

- **Instance Variable:** Variable declared outside any method,block or constructor without using static keyword is called as instance variable.

- **Local Variable:** Variable declared inside any method,block or constructor is called as local variable.
  Note: We can never declare any static or instance variable inside any method,block or constructor.

```
class Sample{
                public static void main(String []args){
```

```
        int a = Integer.parseInt(args[0]);// "100" --> 100
        int b = Integer.parseInt(args[1]);
        int sum = a+b;//pure expression
    System.out.println("Sum of "+a+" and "+b+" : "+sum);
    }
    }
```

public: anyone can access anywhere
static: can be accessed directly without creating object
void: must not return any value
main: contains main logic for application which will be executed
String: input by user can be any type, so compatibiity is for String only
[]: user may input any number of values during execution
args: variable name that matches the readability

Structure of parseInt() in Integer:

```
class Integer{
public static int parseInt(String s){
int value;
value = ....;
return value;
}
}
```

Naming Rules and Convention for variables
In Java, variables must follow certain rules (syntax requirements)
and conventions (recommended best practices) for better readability and consistency.
Naming Rules (Mandatory — Compiler Enforced)
These are the rules you must follow when declaring variable names:

Start with a letter, underscore _, or dollar sign $
Valid: name, _value, $count
Invalid: 1name, #data (cannot start with digit or special characters)

Can contain letters, digits, underscores, or dollar signs
Valid: age1, user_name, total$amountCannot use Java keywords as variable names
Invalid: int, class, new

Variable names are case-sensitive
score and Score are different variables

No spaces allowed in variable names
Invalid: total marks

Naming Conventions (Best Practices — Not Enforced)
These improve readability and follow Java community standards.

Convention	Rule	Example
Use **camelCase** for variable names	First word lowercase, next words capitalized	studentName, totalMarks, isEligible
Use **meaningful** names	Avoid single letters except in small loops	count, employeeSalary, not x, y

Convention	Rule	Example
Avoid using $ and _ unless for special cases	$result and _temp are legal, but not recommended	
Constants should be in **UPPERCASE_SNAKE_CASE**	Use for final variables	MAX_VALUE, DEFAULT_TIMEOUT
Start with a **noun**	Describes the data the variable holds	fileName, price, balance

---

```
// Good examples
        int studentAge = 20;
        double totalAmount = 1500.75;
        boolean isAvailable = true;

        // Not recommended
        int x = 5;        // unclear
        double a1b2c3 = 0; // meaningless
        String StudentName = "Jagannath"; // Should be studentName (not PascalCase for variables)
```

Quiz: Choose if the variable name is Valid or Invalid

```
int myAge = 25;

double $price = 19.99;

String 1stName = "Alice";

boolean isStudent;

float total amount = 500.0f;

int class = 10;

long _salary = 100000;

int Score = 90;

char grade@ = 'A';

final int MAX_VALUE = 1000;
```

No.	Variable Name	Valid?	Why?
1	myAge	✓	Follows camelCase and valid characters
2	$price	✓	$ is allowed (but not recommended)

No.	Variable Name	Valid?	Why?
3	1stName	✗	Cannot start with a digit
4	isStudent	✓	Follows camelCase and descriptive
5	total amount	✗	Spaces not allowed in variable names
6	class	✗	class is a reserved keyword
7	_salary	✓	Starts with _, which is allowed
8	Score	✓	Valid (but conventionally should be score)
9	grade@	✗	@ is not allowed in variable names
10	MAX_VALUE	✓	Conventionally correct for constants

Local Variable:

- The variable which is declared inside any method, block or constructor is called as local variable.
- Whenever we need a memory specific to one particular scope then we must choose local variable.
- Local variables are temporary variables.

e.g to demonstrate local variable cant be accessed outside their scope:

```
class Test{
public static void m1(){
int x = 100;
System.out.println("Value of x: "+x);
}
public static void main(String []args){
System.out.println("main starts");
System.out.println("Value of x from m1() is: "+x);//error: cannot find symbol
System.out.println("main ends");
}
}
```

- Local variables can have same name only when they are declared in different scopes.

```
class Test{
public static void m1(){
int x = 100;
System.out.println("Value of x in m1(): "+x);
}
public static void main(String []args){
System.out.println("main starts");
int x = 500;
System.out.println("Value of x in main: "+x);
System.out.println("main ends");
}
}
```

- Local variables cant have same name if they are declared in same scope, because there will be ambiguity(confusion) for JVM about which one has to be accessed.

- Local variables must be intialized before their utilzation as they dont have default values.If we utilize them before initialization then we will get

compile time error i.e. 'variable __ might not have been initialized'.

```
class Test{
public static void main(String []args){
System.out.println("main starts");
int x;//variable x is declared
System.out.println("Value of x in main: "+x);/*error: variable x might not have been initialized*/
System.out.println("main ends");
}
}
```

- We access local variables directly by their name.

Static Variables:

- The variables which are declared outside any method, block or constructor by using the keyword 'static' are called as static variables.
- Static variables are also called as Class Variables.
- Static variables can be accessed in following ways:
  i). directly in the same class
  ii). using class name from another class
  or whenever SV and LV have same name.
  iii). using reference of an object(not recommended because they are allocated only once for class and not object)

Target: Static variables are accessed directly in the same class.

Flipkart company has announced that there will be Rs500 off for all the users if the billing amount is more than 1000.

```
class FlipkartAccount{
String userId;
String password;
long mobileNo;
String mailId;
String address;
static int bonanzaOffer = 500;
public void login(String userId,String password){
if(userId.equals("ryzen123") && password.equals("octagon")){
System.out.println("Login successful");
System.out.println("Bonanza Offer of Rs."+bonanzaOffer+" for purchase of Rs.1000 & above");
//accessing SV directly in same class
System.out.println("Happy Shopping");
}
else{
System.out.println("Invalid userID or Password.Try Again");
}
}
public static void main(String []args){
FlipkartAccount f1 = new FlipkartAccount();
f1.userId = "ryzen123";
f1.password = "Octagon" ;
f1.mobileNo = 9876098743L;
```

```
f1.mailId = "ryzen_123@gmail.com";
f1.address = "Hyderabad, GareebPet";
f1.login(f1.userId,f1.password);
}
}
```

Can static variable and local variable share the same name?

Answer: Yes static variables and local variable can share the same name. If we have to access local variable then we can access directly but to access the static variable we must use class name because JVM gives the first priority to local variable when static variable and local variable share the same name.

```
class Student{
static int age = 17;
public static void main(String []args){
int age = 20;
System.out.println("Age: "+age);//Local variable will be accessed
System.out.println("Age: "+Student.age);//static variable will be accessed
}
}
```

```
class SBIYono
        {

            String userID;
            String password;
            static String bankName = "SBI";//SBI is assigned to bankName by using =

            public static void main(String[] args)
            {
                SBIYono user1 = new SBIYono();
                SBIYono user2 = new SBIYono();
                System.out.println("Bank Name: "+bankName);
                System.out.println("Before setting any value: ");
                System.out.println("Babuji ka  userID: "+user1.userID);
                System.out.println("Babuji ka  password: "+user1.password);
                System.out.println();
                System.out.println("Mataji ka  userID: "+user2.userID);
                System.out.println("Mataji ka  password: "+user2.password);
                    System.out.println();
                System.out.println("After setting userID and password: ");
                user1.userID = "James Golsing";
                user1.password = "123abc";
                System.out.println("Babuji ka  userID: "+user1.userID);
                System.out.println("Babuji ka  password: "+user1.password);
                user2.userID = "Stephen Robbins";
                user2.password = "Java45";
                System.out.println("Mataji ka  userID: "+user2.userID);
                System.out.println("Mataji ka  password: "+user2.password);
                //System.out.println("User ID: "+userID); error
                //System.out.println("Password: "+password); error
            }
        }
```

- Static variables and instance variables of a class can be accessed in the same class as well as from another class.

- Local variables are accessed only within the scope where they are created.
- Static variables are accessed directly in the same class where they are created.
- We can't access instance variable/ non static variable directly from a static method.
- We can access the instance variables by the help of object.
- To create an object we use 'new' keyword.

```
class BankAccount{
 String userName;
 int age;
 long accountNo;
 double balance;
 long contactNo;
 public static void main(String []args){
 BankAccount sbi = new BankAccount();
 BankAccount icici = new BankAccount();
 BankAccount hdfc = new BankAccount();
 sbi.userName = "Jagannath";
 sbi.age = 20;
 sbi.accountNo = 1234567890L;
 sbi.balance = 1900.8;
 sbi.contactNo = 9876543210L;

 icici.userName = "James";
 icici.age = 21;
 icici.accountNo = 45687654567890L;
 icici.balance = 30900.70;
 icici.contactNo = 93472367899L;

 hdfc.userName = "Robert";
 hdfc.age = 27;
 hdfc.accountNo = 87656782324210L;
 hdfc.balance = 90.8;
 hdfc.contactNo = 6370987342L;

 System.out.println("SBI Account details: ");
 System.out.println("Name of user: "+sbi.userName);
 System.out.println("Age         :"+sbi.age);
 System.out.println("Account no. : "+sbi.accountNo);
 System.out.println("Balance     : "+sbi.balance);
 System.out.println("Contact no. : "+sbi.contactNo);
 System.out.println();
 System.out.println("HDFC Account details: ");
 System.out.println("Name of user: "+hdfc.userName);
 System.out.println("Age         : "+hdfc.age);
 System.out.println("Account no. : "+hdfc.accountNo);
 System.out.println("Balance     : "+hdfc.balance);
 System.out.println("Contact no. : "+hdfc.contactNo);
 System.out.println();
 System.out.println("ICICI Account details: ");
```

```
 System.out.println("Name of user: "+icici.userName);
 System.out.println("Age         : "+icici.age);
 System.out.println("Account no. : "+icici.accountNo);
 System.out.println("Balance     : "+icici.balance);
 System.out.println("Contact no. : "+icici.contactNo);
 }
}
```

- class name must start with uppercase letter
- we can use multiple words as a single class name but make sure each and every word's initial must be in uppercase

Question 1:
A company maintains employee records where all employees work for the same company, but
each has a unique ID and name.

Task:
Create a class Employee with:
A static variable companyName initialized to "TechCorp".
Non-static variables: empId and empName.
Directly assign values to variables inside the main method using object.
Print employee details and observe the static variable behavior.
Expected Output:
Changing companyName should affect all employees.
Each employee should have a different ID and name.

Question 2:
A bank maintains customer accounts where all accounts share the same interest rate, but each account has a different account number and balance.

Task:
Create a class BankAccount with:
A static variable interestRate initialized to 7.5.
Non-static variables: accountNumber and balance.
Assign values directly inside the main method.
Print account details and modify interestRate to see its effect on all accounts.
Expected Output:
Each account has a unique number and balance.
Changing interestRate should reflect for all accounts.

Question 3:
An e-commerce system tracks customer orders. Each order has a unique ID and amount, but the system keeps count of total orders.

Task:
Create a class Order with:
A static variable totalOrders initialized to 0.
Non-static variables: orderId and orderAmount.
Increase totalOrders manually inside the main method when an order is placed.
Print order details and observe the static variable behavior.
Expected Output:
totalOrders increases when a new order is created.

Each order has a unique ID and amount.

Question 4:
A flight booking system tracks passengers. Each passenger has a unique ID and name, but the system maintains a count of total passengers booked.

Task:
Create a class Passenger with:
A static variable totalPassengers initialized to 0.
Non-static variables: passengerId and passengerName.
Increase totalPassengers manually in the main method when a new passenger is added.
Print passenger details and check the total number of passengers booked.
Expected Output:
Each passenger has a unique ID and name.
totalPassengers increases for each new passenger.

Question 5:
A bank allows customers to deposit money, withdraw money, and check their current balance.
The bank keeps track of the total number of transactions performed across all accounts.

Requirements:
Create a class BankAccount with:
A static variable totalTransactions to track the total number of transactions across all accounts.
A non-static variable balance to store each account's balance.
Implement three methods:
public void deposit(int amount): Increases the balance and updates totalTransactions.
public void withdraw(int amount): Decreases the balance if sufficient funds are available and updates totalTransactions.
checkBalance(): Displays the current balance.
In the main method:
Create multiple accounts.
Perform deposit and withdrawal operations.
Display the balance for each account.
Display the total transactions performed across all accounts.

PRACTISE THE FOLLOWING:
1. Which of the following keyword is used to create an instance of a class?
a.new
b.public
c.class
d.main
Answer:

2. The _____ is called as an instance of a class.
a. state
b. object
c. attribute
d. features

3. The objects we see around are called as _____
a. real world objects

b. virtual objects
c. imaginary objects
d. none of the above

4. Which of the following makes a method non-returnable?
a. public
b. static
c. void
d. new

4. Which of the following statements is valid for the object?
a. They possess same characteristics and behaviour
b. they posses same characteristics but different behaviour
c. they posses different characteristics and different behaviour
d. they posses different characteristics but common behaviour

5. Creating _____ is the fundamental concept in OOP.
a. variable
b. method
c. constructor
d. class
e. object

6. A class is also considered as an _____ factory.
a. object
b. variable
c. constructor
d. method

7. A real world object deals with characteristics and _____.
a. behaviour
b. attributes
c. constructors
d. sub classes

8. The _____ of a class differs on various characteristics.
a.object
b.variable
c.method
d.constructor

9. The characteristics of the _____ objects are considered to the data members(variables) of the _____ objects.
a. real world,software
b. instance, static
c. private,public
d. virtual,imaginary

10. An object of a class is created by using a keyword_____.
a. class
b. public
c. new
d. return

11. How many bytes a char data type occupies in the memory?

a. 2
b. 8
c. 4
d. 16

12. Which of the following is a non-primitive(Referenced) data type?
a. int
b. double
c. char
d. String

13. Which of the following is a non-primitive data type?
a. int
b. Byte
c. char
d. byte

14. Which of the following is non-numeric data type?
a. boolean
b. float
c. int
d. byte

15. Which of the following ASCII range is applicable for lowercase letters?
a. 65-90
b. 90-115
c. 97-122
d. 95-110

16. What is not an escape sequence?
a. \t
b. \\
c. \n
d. ||

17. Give reasons whether the following assignments are correct or not.
a. int m = 127;
b. float f = 0.0026534567;
c. String str = 'Jagannath';
d. boolean p = False;
e. String b = "true";
f. char ch = "java";
g. String st = "Java by Jagannath";
h. double n = 454.14567654;

18. Given x+ = x++ + ++x + --x
Find the value when x = 5.

19. Who developed java?
a. James Reddy
b. James Gosling
c. James Robert
d. James Bhandari

20. Which of the following is not a java reserved word?
a. private
b. public
c. new
d. New

Q1. Write a program for the following requirement:
StudyTable is an object of the class Furniture.
Provide the operations and data members of your choice.
Print the details with suitable message.

Q2. Design a class MobilePhone with certain attributes and methods.
Display the details of different mobile phones with suitable message.

Q3. Design a class Bike with certain attributes and methods.
Display the details of different mobile phones with suitable message.

Q4. Design a class Fruits with certain attributes and methods.
Display the details of different mobile phones with suitable message.

Q5. Design a class Pen with certain attributes and methods.
Display the details of different pens with suitable message.

Q6. Write a program to calculate the discount given to a customer on purchase of a LEDTelevision. The program also displays the amount paid by the customer after disount.The details are given as:
Class name: Television
Data Members: cost,discount,amount
Member methods:
accept(): store the cost of the TV
calculate(): calculate the discount
display(): show the discount and the amount to be paid after discount.
" Its your TV so you decide the discount."

1. **Can we access Local variables from another method?**
   Answer: No we cant access local variables from another method because local variables are temporary variables and they are restricted within the scope where they are created.

   2. Can we access static variables from a non static method?
   Answer: Yes we can access static variables from a non static method directly or by using class name.

   3. Can we access static variable from a static method?
   Answer: Yes we can access static variables from a static method directly or by using class name.

4. Can we access non static vaiables from a non static method?
Answer: Yes we can access non static variables from a non static method directly or by object.

5. Can we access non static variable from a static method?
Answer: Yes we can access non static variables from a static method by using object.

--------- Method in Java ---------

- What is method?
Answer: Method is a part of program which has a body in which we provide business logic so that it can be reused later.Method is used to perform operations(business operations).

Where do we use method?
Answer: We use a method inside a class,interface,enum,innerclass.

Why do we use method?
Answer: We use a method to perform business operations and also re-use a set of code anywhere in the program.

How do we declare a method?
Answer: To declare a method we must follow the given syntax:

```
<access modifier>  <non access modifier>  <return type>  <method name>  (parameters)
{
   logic;
};
```

How do we execute a method?
Answer: We execute a method by calling it from main method.

How many methods can be declared in a program?
Answer: Depends on project requirement.

How many time we can execute a method?
Answer: Depends on requirement.

How many types of methods are there?
Answer: Methods are categorised into 5 types depending on different factors:
- based on declaration
- based on accessibility
- based on input
- based on return
- based on body/implementation

*** Based on declaration, method is of two types:**
- Predefined Method
- User Defined Method

*** Based on accessibility, method is of two types:**
- static method
- non-static method

*** Based on input, method is of two types:**
- parameterized method
- non parameterized method

*** Based on return,method is of two types:**
- method with data type as return type
- method with void as return type

*** Based on body/implementation, method is of two types:**
- concrete/complete method
- abstract/incomplete method (abstract methods will be discussed in Abstraction)

Scenario 1: Salary Calculation Using Arithmetic and Relational Operators
Problem Statement:
XYZ Pvt. Ltd. wants to automate its **salary calculation system**. The company has set a **flat tax rate** applicable to all employees, which should be stored as a **static variable**. Each employee has a **basic salary** and a **bonus**, which are different for each employee and should be stored as **instance variables**.

The final salary after tax deduction is calculated using the formula:

$$\text{Final Salary} = (\text{Basic Salary} + \text{Bonus}) - \left(\frac{\text{Tax Rate} \times (\text{Basic Salary} + \text{Bonus})}{100}\right)$$

The company wants to check if the **final salary is greater than ₹50,000** using a **relational operator** and display an appropriate message:
- If the salary is **above ₹50,000**, display **"Eligible for premium benefits."**
- Otherwise, display **"Eligible for standard benefits."**

Tasks to Perform:
1. Declare a **static variable** for the **tax rate**.
2. Declare **instance variables** for **basic salary** and **bonus**.
3. Use **arithmetic operators** to compute the final salary.
4. Use a **relational operator** to compare the salary.
5. Print the **final salary** and the **benefits eligibility message**.

Scenario 2: Scholarship Eligibility Using Logical Operators
Problem Statement:
A university wants to automate its **scholarship eligibility** process. A student is eligible for a scholarship **only if**:
 Their **attendance is 75% or above**.
 Their **average marks are 85 or above**.
 If either condition fails, the student is **not eligible**.

The **minimum passing marks** for all students is **40**, which should be stored as a **static variable**. Each student has different **marks** and **attendance**, stored as **instance variables**.
Tasks to Perform:
1. Declare a **static variable** for the **minimum passing marks**.
2. Declare **instance variables** for **average marks** and **attendance percentage**.
3. Use **logical operators (&&, ||)** to check eligibility conditions.
4. Display a message stating whether the student qualifies for the scholarship.

Scenario 3: Car Rental System Using Unary and Assignment Operators
Problem Statement:
A **car rental company** manages the number of available cars and tracks its revenue. The company charges different amounts for different cars. When a car is rented:
 The **total cars available** should be decremented using the **unary operator (--)**.
 The **total revenue** should be increased using the **assignment operator (+=)**.
 The **total number of cars in the system** is a **static variable**.
 Each rental transaction involves a **rental charge**, stored as an **instance variable**.

Tasks to Perform:
1. Declare a **static variable** for the **total number of available cars**.
2. Declare an **instance variable** for the **rental charge per customer**.
3. When a car is rented:
    - Use the **unary operator (--)** to decrease the car count.
    - Use the **assignment operator (+=)** to add the rental charge to total revenue.
4. Display the **updated car count** and **total revenue** after each rental.

Scenario 4: Employee Access Control Using Bitwise Operators
Problem Statement:
A **company's security system** grants access to employees based on their **security level code**. Each employee has a **unique security code**, which is checked against a predefined **company security access code** (stored as a **static variable**).

- If an employee's security code **matches** the company's access code using the **bitwise AND (&)** operation, they are **granted access**.
- If the access code does not match, they are **denied access**.
- If access is denied, use a **bitwise OR (|)** operation to modify the access code.

Tasks to Perform:
1. Declare a **static variable** for the **company security access code**.
2. Declare an **instance variable** for the **employee security code**.
3. Use **bitwise AND (&)** to check if access is granted.
4. If access is denied, use **bitwise OR (|)** to modify the employee's security code.
5. Display a message **granting or denying access**.

Scenario 5: Loan Approval Using Ternary Operator
Problem Statement:

A **bank's loan approval system** determines whether a customer is eligible for a loan based on their **cr score**.

- The **minimum required cr score** is **700**, which should be stored as a **static variable**.
- Each customer has a **different cr score**, stored as an **instance variable**.
- Use the **ternary operator (? :)** to check whether the loan is approved:
  - If the cr score is **700 or above**, print **"Loan Approved"**.
  - Otherwise, print **"Loan Denied"**.

Tasks to Perform:

1. Declare a **static variable** for the **minimum cr score**.
2. Declare an **instance variable** for the **customer's cr score**.
3. Use a **ternary operator (? :)** to determine loan approval.
4. Display a message stating **whether the loan is approved or denied**.

Pre Defined Methods:

- The methods which are already available in the technology are called as predefined methods.
- The predefined methods are already created inside predefined classes such as: Math,String,StringBuffer,StringBuilder,Arrays,Byte,Short,Integer, Float,Long,Double,Character,Boolean,Number,Throwable,Exception,Error, Object,Scanner,System,Collections etc.
- The predefined methods are also available in predefined interfaces such as Collection,List,Set,Queue,Map,Iterator,Iterable,Predicate, Consumer,Supplier,Function etc.

(*** The predefined classes and interfaces are available inside predefined packages such as: lang,util,io,sql,txt etc.
*** The predefined packages are avaialable inside API such as 'java')

- We dont have to create predefined methods as they are already defined.We just have to invoke them wherever it is required.
- Predefined methods were introduced to design common operations of real world project.

Some of the common methods in java.lang.Math:

- public static double sqrt(double):

This method returns the square root of passed argument.

e.g:

```
//WAP to store a number and display the square root of the mumber
class TestMethod1{
public static void main(String []args){
int num = Integer.parseInt(args[0]);
double result = Math.sqrt(num);
System.out.println("Square root of "+num+" is: "+result);
}
}

javac TestMethod1.java
java TestMethod1 25
//Output: Square root of 25 is: 5.0
```

- We can't get the square root of a negative number.If we try we will get NaN.

- **public static double cbrt(double):**
  This method returns the cube root of passed argument.
   e.g:
   class TestMethod2{
  public static void main(String []args){
  int num = Integer.parseInt(args[0]);
  double result = Math.cbrt(num);
  System.out.println("Square root of "+num+" is: "+result);
  }
}
javac TestMethod2.java
java TestMethdod2 27
Output: Cube root of 27.0 is 3.0

Math.ceil(double)

Returns the smallest double value that is greater than or equal to the argument and equal to a mathematical integer.

```
double price = 99.3;
System.out.println("Ceil of " + price + " = " + Math.ceil(price)); // 100.0
```

Math.floor(double)

Returns the largest double value that is less than or equal to the argument and equal to a mathematical integer.

```
double height = 175.9;
System.out.println("Floor of " + height + " = " + Math.floor(height)); // 175.0
```

Math.round(float)

Returns the closest int to the argument, with ties rounding up.

```
float rating = 4.6f;
System.out.println("Rounded float = " + Math.round(rating)); // 5
```

Math.round(double)

Returns the closest long to the argument, with ties rounding up.

```
double balance = 999.7;
System.out.println("Rounded double = " + Math.round(balance)); // 1000
```

Math.random()

Returns a double value with a positive sign, greater than or equal to 0.0 and less than 1.0.

```
System.out.println("Random value = " + Math.random()); // e.g., 0.48762334
```

Math.abs(int)

Returns the absolute value of an int value (removes negative sign).

```
int temp = -10;
System.out.println("Absolute value = " + Math.abs(temp)); // 10
```

Math.max(int, int)

Returns the greater of two int values.

```
int a = 20, b = 45;
System.out.println("Max = " + Math.max(a, b)); // 45
```

Math.max(long, long)

Returns the greater of two long values.

```
long salary1 = 50000L, salary2 = 70000L;
System.out.println("Max Salary = " + Math.max(salary1, salary2)); // 70000
```

Math.max(float, float)

Returns the greater of two float values.

```
float percentage1 = 88.5f, percentage2 = 91.2f;
System.out.println("Max Percentage = " + Math.max(percentage1, percentage2)); // 91.2
```

Math.max(double, double)

Returns the greater of two double values.

```
double cgpa1 = 8.25, cgpa2 = 8.5;
System.out.println("Max CGPA = " + Math.max(cgpa1, cgpa2)); // 8.5
```

Math.min(int, int)

Returns the smaller of two int values.

```
int age1 = 20, age2 = 25;
System.out.println("Min Age = " + Math.min(age1, age2)); // 20
```

Math.min(long, long)

Returns the smaller of two long values.

```
long popA = 150000L, popB = 120000L;
System.out.println("Min Population = " + Math.min(popA, popB)); // 120000
```

Math.min(float, float)

Returns the smaller of two float values.

```
float heightA = 5.7f, heightB = 5.2f;
System.out.println("Min Height = " + Math.min(heightA, heightB)); // 5.2
```

Math.min(double, double)

Returns the smaller of two double values.

```
double weight1 = 62.3, weight2 = 58.9;
System.out.println("Min Weight = " + Math.min(weight1, weight2)); // 58.9
```

Predefined Methods of **java.lang.String:-**

charAt(int index)
- compareTo(String anotherString)
- compareToIgnoreCase(String str)
- concat(String str)
- contains(CharSequence s)
- endsWith(String suffix)
- equals(Object anObject)
- equalsIgnoreCase(String anotherString)
- getBytes()
- indexOf(int ch)

- indexOf(int ch, int fromIndex)
- indexOf(String str)
- indexOf(String str, int fromIndex)
- isEmpty()
- lastIndexOf(int ch)
- lastIndexOf(int ch, int fromIndex)
- lastIndexOf(String str)
- lastIndexOf(String str, int fromIndex)
- length()
- replace(char oldChar, char newChar)
- split(String regex)
- strip()
- stripLeading()
- stripTrailing()
- substring(int beginIndex)
- substring(int beginIndex, int endIndex)
- toCharArray()
- toLowerCase()
- toUpperCase()
- trim()

e.g:

1. length()

Returns the number of characters present in the string.

Return Type: int
Use: To find the length of a name, sentence, etc.

```
String name = "Jagannath";
System.out.println("Length = " + name.length()); // 9
```

2. charAt(int index)

Returns the character at the specified index.

Return Type: char
Use: Useful when validating first letter or accessing characters.

```
String name = "Jagannath";
System.out.println("Character at index 0 = " + name.charAt(0)); // J
```

3. equals(String other)

Compares this string to the specified object.

Return Type: boolean
Use: To compare login credentials or inputs.

```
String input = "Jagannath";
System.out.println(input.equals("Jagannath")); // true
```

4. equalsIgnoreCase(String other)
Compares two strings, ignoring case considerations.

Return Type: boolean
Use: Case-insensitive form inputs.

```
String city = "delhi";
System.out.println(city.equalsIgnoreCase("Delhi")); // true
```

5. compareTo(String anotherString)
Compares two strings lexicographically.

Return Type: int
Use: For alphabetical sorting.

```
String a = "Apple", b = "Banana";
System.out.println(a.compareTo(b)); // negative value
```

6. substring(int beginIndex)
Returns a new string that is a substring from beginIndex to end.

Return Type: String
Use: Extracting portions like domain, name parts.

```
String fullName = "Jagannath Bhandari";
System.out.println(fullName.substring(10)); // Bhandari
```

7. substring(int beginIndex, int endIndex)
Returns a new string from beginIndex to endIndex - 1.

Return Type: String
Use: Extract first or last name, etc.

```
String fullName = "Jagannath Bhandari";
System.out.println(fullName.substring(0, 9)); // Jagannath
```

8. contains(CharSequence seq)
Checks whether sequence of characters exists in the string.

Return Type: boolean
Use: To check if email contains '@gmail'.

```
String email = "jagannath@gmail.com";
System.out.println(email.contains("@gmail")); // true
```

9. replace(char oldChar, char newChar)
Returns a new string resulting from replacing all occurrences of oldChar with newChar.

Return Type: String
Use: Modify text like replacing spaces or characters.

```
String msg = "Java is fun";
System.out.println(msg.replace("a", "@")); // J@v@ is fun
```

10. toUpperCase()
Converts all characters to uppercase.

Return Type: String
Use: Format user inputs.

```
String name = "jagannath";
System.out.println(name.toUpperCase()); // JAGANNATH
```

11. toLowerCase()

Converts all characters to lowercase.

Return Type: String
Use: Normalize user input for comparison.

```
String name = "JAGANNATH";
System.out.println(name.toLowerCase()); // jagannath
```

12. trim()

Removes leading and trailing whitespaces.

Return Type: String
Use: Clean user input or form data.

```
String input = "  Jagannath  ";
System.out.println(input.trim()); // Jagannath
```

13. split(String regex)

Splits the string around matches of the given regex.

Return Type: String[]
Use: Break input like CSV or full name.

```
String data = "Jagannath,20,M";
String[] arr = data.split(",");
System.out.println(arr[0]); // Jagannath
```

14. indexOf(String str)

Returns the index of the first occurrence of the substring.

Return Type: int
Use: Find location of '@' in email.

```
String email = "jagannath@gmail.com";
System.out.println(email.indexOf("@")); // 9
```

15. lastIndexOf(String str)

Returns the index of the last occurrence of the substring.

Return Type: int
Use: Find position of last dot in filename.

```
String fileName = "report.final.version.pdf";
System.out.println(fileName.lastIndexOf(".")); // 20
```

16. startsWith(String prefix)

Checks if string starts with the given prefix.

Return Type: boolean
Use: Validate email or name start.

```
String name = "Jagannath";
System.out.println(name.startsWith("Jaga")); // true
```

17. endsWith(String suffix)

Checks if string ends with the given suffix.

Return Type: boolean
Use: Validate file extensions or domain.

```
String file = "document.pdf";
System.out.println(file.endsWith(".pdf")); // true
```

18. isEmpty()

Checks if the string is empty (length == 0).

Return Type: boolean
Use: Form validation.

```
String input = "";
System.out.println(input.isEmpty()); // true
```

19. join(CharSequence delimiter, CharSequence... elements)

Joins multiple strings using a delimiter.

Return Type: String
Use: Construct CSV or full name.

```
String joined = String.join("-", "Jagannath", "20", "M");
System.out.println(joined); // Jagannath-20-M
```

20. valueOf(int i)

Converts int to String representation.

Return Type: String
Use: Convert numeric input to string for display.

```
int age = 20;
String ageStr = String.valueOf(age);
System.out.println(ageStr); // 20
```

Practise

Real-Time Examples of String Methods in Java
==========

1. charAt(int index)

Use: To get the first initial from designation.
Example:

```
String designation = "Software Developer";
char initial = designation.charAt(0); // 'S'
```

2. compareTo(String anotherString)

Use: Compare two names alphabetically.
Example:

```
String emp1 = "Jagannath";
String emp2 = "Abhijit";
```

System.out.println(emp1.compareTo(emp2)); // > 0 (because 'J' > 'A')

3. compareToIgnoreCase(String str)
Use: Case-insensitive name comparison.
Example:
String a = "Jagannath", b = "jagannath";
System.out.println(a.compareToIgnoreCase(b)); // 0 (equal ignoring case)

4. concat(String str)
Use: Merge first and last name.
Example:
String fullName = "Jagannath ".concat("Bhandari"); // "Jagannath Bhandari"

5. contains(CharSequence s)
Use: Check if email contains "gmail".
Example:
String email = "jagannath@gmail.com";
System.out.println(email.contains("gmail")); // true

6. endsWith(String suffix)
Use: Check if file ends with .pdf.
Example:
String file = "resume.pdf";
System.out.println(file.endsWith(".pdf")); // true

7. equals(Object anObject)
Use: Verify login name.
Example:
String stored = "Jagannath";
System.out.println(stored.equals("Jagannath")); // true

8. equalsIgnoreCase(String anotherString)
Use: Ignore case in login comparison.
Example:
String name = "JAGANNATH";
System.out.println(name.equalsIgnoreCase("jagannath")); // true

9. getBytes()
Use: Store string as bytes.
Example:
String data = "Jagannath";
byte[] bytes = data.getBytes();
System.out.println(bytes.length); // 8

10. indexOf(int ch)
Use: Find '@' in email.
Example:
String email = "jagannath@gmail.com";
System.out.println(email.indexOf('@')); // 9

11. indexOf(int ch, int fromIndex)
Use: Find next space after first name.
Example:
String info = "Jagannath Bhandari Developer";

```
System.out.println(info.indexOf(' ', 10)); // 19
```

12. indexOf(String str)
Use: Check if role includes "Developer".
Example:
```
String job = "Software Developer";
System.out.println(job.indexOf("Developer")); // 9
```

13. indexOf(String str, int fromIndex)
Use: Find second "Java".
Example:
```
String msg = "Java is easy. Java is powerful.";
System.out.println(msg.indexOf("Java", 5)); // 15
```

14. isEmpty()
Use: Check if search box is empty.
Example:
```
String input = "";
System.out.println(input.isEmpty()); // true
```

15. lastIndexOf(int ch)
Use: Last dot in file path.
Example:
```
String path = "C:/files/jagannath_resume.pdf";
System.out.println(path.lastIndexOf('.')); // 24
```

16. lastIndexOf(int ch, int fromIndex)
Use: Last space before index 15.
Example:
```
String job = "Software Developer Engineer";
System.out.println(job.lastIndexOf(' ', 15)); // 8
```

17. lastIndexOf(String str)
Use: Last "Error" in logs.
Example:
```
String log = "Error: login failed. Error: timeout.";
System.out.println(log.lastIndexOf("Error")); // 25
```

18. lastIndexOf(String str, int fromIndex)
Use: Last "Error" before position 20.
Example:
```
String log = "Error: login failed. Error: timeout.";
System.out.println(log.lastIndexOf("Error", 20)); // 0
```

19. length()
Use: Length of full name.
Example:
```
String fullName = "Jagannath Bhandari";
System.out.println(fullName.length()); // 19
```

20. replace(char oldChar, char newChar)
Use: Mask digits in phone number.
Example:
```
String phone = "9876543210";
```

```
System.out.println(phone.replace('9', 'X')); // X876543210
```

21. split(String regex)
Use: Split CSV data.
Example:
```
String data = "Jagannath,20,M,Software Developer";
String[] parts = data.split(",");
System.out.println(parts[3]); // Software Developer
```

22. strip()
Use: Remove extra spaces.
Example:
```
String name = "  Jagannath  ";
System.out.println(name.strip()); // Jagannath
```

23. stripLeading()
Use: Clean left padding.
Example:
```
String s = "   Jagannath";
System.out.println(s.stripLeading()); // "Jagannath"
```

24. stripTrailing()
Use: Clean right padding.
Example:
```
String s = "Jagannath   ";
System.out.println(s.stripTrailing()); // "Jagannath"
```

25. substring(int beginIndex)
Use: Get domain from email.
Example:
```
String email = "jagannath@gmail.com";
System.out.println(email.substring(10)); // gmail.com
```

26. substring(int beginIndex, int endIndex)
Use: Get first name from full name.
Example:
```
String fullName = "Jagannath Bhandari";
System.out.println(fullName.substring(0, 9)); // Jagannath
```

27. toCharArray()
Use: Convert password to char array.
Example:
```
String password = "Jag@123";
char[] chars = password.toCharArray();
System.out.println(chars[0]); // 'J'
```

28. toLowerCase()
Use: Normalize usernames.
Example:
```
String user = "JAGANNATH";
System.out.println(user.toLowerCase()); // jagannath
```

29. toUpperCase()
Use: For ID printing.
Example:

```
String user = "jagannath";
System.out.println(user.toUpperCase()); // JAGANNATH
```

30. trim()

Use: Clean user input.
Example:

```
String input = "  Jagannath ";
System.out.println(input.trim()); // Jagannath
```

Solve the following:

1. Sum of Two Numbers:
Write a method addNumbers(int a, int b) that takes two integers as parameters and returns their sum.

2. Check Even or Odd:
Write a method isEven(int num) that takes an integer as input and returns true if the number is even, otherwise false.

3. Factorial Calculation:
Write a method calculateFactorial(int num) that takes an integer as input and returns its factorial.

4. Find Maximum of Three Numbers:
Write a method findMax(int a, int b, int c) that returns the largest among three numbers.

5. Check Prime Number:
Write a method isPrime(int num) that checks if a given number is prime.

6. Reverse a String:
Write a method reverseString(String str) that takes a string and returns its reverse.

7. Count Vowels in a String:
Write a method countVowels(String str) that counts the number of vowels in a given string.

8. Check Palindrome:
Write a method isPalindrome(String str) that checks whether a given string is a palindrome.

9. Greatest Common Divisor (GCD):
Write a method findGCD(int a, int b) that calculates the GCD of two numbers.

10. Fibonacci Series:
Write a method generateFibonacci(int n) that prints the first n Fibonacci numbers.

11. Armstrong Number:
Write a method isArmstrong(int num) to check if a number is an Armstrong number.

12. Sorting an Array:
Write a method sortArray(int[] arr) that takes an array of integers and returns the sorted array.

13. Find Duplicate Elements in an Array:
Write a method findDuplicates(int[] arr) that prints duplicate elements in an array.

14. Calculate Compound Interest:
Write a method calculateCompoundInterest(double principal, double rate, int time) that returns the compound interest.

15. Validate Email Format:
Write a method isValidEmail(String email) that checks whether a given email is valid using basic validation rules.

Q. How do we take input from user?
Answer: Taking input from the user:
- using command line argument
- using methods of Scanner class

java.util.Scanner:

- It is a predefined class.
- It is available in java.util package.
- It has to be imported into the program where we use it.
- To import Scanner into the program we need to write import statement:
  import java.util.Scanner;
  import- bring, java – API, util – package, Scanner - class
  The above statement means we are directing the system to allow the developer to utilize the functionalities of Scanner in the current program.

- Users are allowed to enter any type of data by the help of methods in Scanner, and the methods are non static methods which are specific to type of data:

Method	Description	Return Type
next()	Reads next token (word)	String
nextLine()	Reads entire line	String
nextInt()	Reads next token as an int	int
nextDouble()	Reads next token as a double	double
nextFloat()	Reads next token as a float	float
nextLong()	Reads next token as a long	long
nextShort()	Reads next token as a short	short
nextByte()	Reads next token as a byte	byte
hasNext()	Returns true if another token is available	boolean
hasNextInt()	Returns true if next token is int	boolean
hasNextDouble()	Returns true if next token is double	boolean
hasNextLine()	Returns true if another line is available	Boolean
useDelimiter(String)	Sets custom delimiter	Scanner
close()	Closes the scanner	void

NOTE: In Java, the **buffer problem** in Scanner usually happens when we **mix different input methods**, especially:
nextInt(), nextDouble(), nextFloat(), etc. with nextLine()

Why Does the Buffer Problem Occur?

Methods like nextInt() **only read the number**, **not the newline character** (\n) left in the input buffer.
So when you later call nextLine(), it **consumes the leftover newline**, not the actual intended input.

Problem Demo:

```
import java.util.Scanner;

        public class BufferProblemExample {
            public static void main(String[] args) {
                Scanner sc = new Scanner(System.in);

                System.out.print("Enter your age: ");
                int age = sc.nextInt(); // Reads only the int, leaves newline in buffer

                System.out.print("Enter your name: ");
                String name = sc.nextLine(); // This reads leftover newline (not name)

                System.out.println("Name: " + name);
                System.out.println("Age: " + age);

                sc.close();
            }
        }
```

Output (if input is):

```
Enter your age: 20
Enter your name:
Name:          ← (blank)
Age: 20
```

Correct Solution – Add sc.nextLine() After nextInt():

```
import java.util.Scanner;

        public class BufferFixedCode {
            public static void main(String[] args) {
                Scanner sc = new Scanner(System.in);

                System.out.print("Enter your age: ");
                int age = sc.nextInt();
                sc.nextLine(); // ☜ This clears the leftover newline

                System.out.print("Enter your name: ");
                String name = sc.nextLine(); // Now it works correctly

                System.out.println("Name: " + name);
                System.out.println("Age: " + age);

                sc.close();
            }
        }
```

Output:

```
Enter your age: 20
Enter your name: Jagannath
Name: Jagannath
Age: 20
```

Where to Apply the Fix?
Apply sc.nextLine(); after any of these**:**
nextInt(), nextDouble(), nextFloat(), nextLong(), nextShort(), nextByte(), nextBoolean()
if you are going to use nextLine() next.

Examples of Scanner methods:

1. next()
```
Scanner sc = new Scanner(System.in);
System.out.print("Enter your name: ");
String name = sc.next();  // Input: Jagannath Bhandari => Reads only "Jagannath"
System.out.println("Hello " + name);
```

2. nextLine()
```
Scanner sc = new Scanner(System.in);
System.out.print("Enter your full name: ");
String fullName = sc.nextLine();  // Input: Jagannath Bhandari => Reads whole line
System.out.println("Hello " + fullName);
```

3. nextInt()
```
Scanner sc = new Scanner(System.in);
System.out.print("Enter your age: ");
int age = sc.nextInt();  // Input: 20
System.out.println("Age: " + age);
```

4. nextDouble()
```
Scanner sc = new Scanner(System.in);
System.out.print("Enter your GPA: ");
double gpa = sc.nextDouble();  // Input: 9.5
System.out.println("GPA: " + gpa);
```

5. nextFloat()
```
Scanner sc = new Scanner(System.in);
System.out.print("Enter your height: ");
float height = sc.nextFloat();  // Input: 5.9
System.out.println("Height: " + height);
```

6. nextLong()
```
Scanner sc = new Scanner(System.in);
System.out.print("Enter phone number: ");
long phone = sc.nextLong();  // Input: 9876543210
System.out.println("Phone: " + phone);
```

7. nextShort()
```
Scanner sc = new Scanner(System.in);
System.out.print("Enter short value: ");
short s = sc.nextShort();  // Input: 1234
System.out.println("Short: " + s);
```

8. nextByte()
```
Scanner sc = new Scanner(System.in);
System.out.print("Enter byte value: ");
byte b = sc.nextByte();  // Input: 120
```

```
System.out.println("Byte: " + b);
```

9. hasNext()

```
Scanner sc = new Scanner("Jagannath 20");
while (sc.hasNext()) {
    System.out.println(sc.next());
}
```

10. hasNextInt()

```
Scanner sc = new Scanner("25 Jagannath");
if (sc.hasNextInt()) {
    int value = sc.nextInt();
    System.out.println("It's an int: " + value);
}
```

11. hasNextLine()

```
Scanner sc = new Scanner("Hello\nJagannath");
while (sc.hasNextLine()) {
    System.out.println(sc.nextLine());
}
```

12. useDelimiter()

```
Scanner sc = new Scanner("Apple,Orange,Banana");
sc.useDelimiter(",");
while (sc.hasNext()) {
    System.out.println(sc.next());
}
```

13. close()

```
Scanner sc = new Scanner(System.in);
//... operations
sc.close();
```

14. nextBoolean()

Reads the next token and interprets it as a boolean value (true or false). Throws InputMismatchException if the input is not valid boolean.

```
import java.util.Scanner;
public class BooleanExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Are you above 18? (true/false): ");
        boolean isAdult = sc.nextBoolean();  // Input: true
        System.out.println("User is adult: " + isAdult);
        sc.close();
    }
}
```

15.

Reading char using Scanner (No nextChar() method exists)

Note: There is no nextChar() method in Scanner, but you can use one of these:

Method 1: Using next().charAt(0)

```
import java.util.Scanner;
public class CharExample {
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your gender (M/F): ");
        char gender = sc.next().charAt(0);  // Reads 1st character of input
        System.out.println("Gender: " + gender);
        sc.close();
    }
}
```

Method 2: Using nextLine().charAt(index)
If you want to read a char from full line (e.g., after using other next*() calls), use:
char ch = sc.nextLine().charAt(0);

Summary Table

Data Type	Method	Example Input	Notes
String	next(), nextLine()	"Hello"	next() ignores space, nextLine() reads full line
int	nextInt()	20	Reads integer
double	nextDouble()	10.5	Reads decimal
float	nextFloat()	5.5f	Use f for float
long	nextLong()	9876543210	Reads long number
short	nextShort()	12345	Reads short
byte	nextByte()	127	Reads byte
boolean	nextBoolean()	true/false	Reads boolean
char	next().charAt(0)	'M'	No direct method

Program : Basic Input Methods

```
import java.util.Scanner;

public class BasicInputScanner {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.next(); // Reads only one word (stops at space)
        System.out.println("You entered: " + str);

        System.out.print("Enter a full line: ");
        sc.nextLine(); // Consume the leftover newline
        String line = sc.nextLine(); // Reads the full line
        System.out.println("Full line: " + line);

        System.out.print("Enter an integer: ");
        int i = sc.nextInt();
        System.out.println("Integer: " + i);

        System.out.print("Enter a float: ");
        float f = sc.nextFloat();
        System.out.println("Float: " + f);

        System.out.print("Enter a double: ");
```

```
        double d = sc.nextDouble();
        System.out.println("Double: " + d);

        System.out.print("Enter a long: ");
        long l = sc.nextLong();
        System.out.println("Long: " + l);

        System.out.print("Enter a boolean (true/false): ");
        boolean b = sc.nextBoolean();
        System.out.println("Boolean: " + b);

        System.out.print("Enter a short: ");
        short s = sc.nextShort();
        System.out.println("Short: " + s);

        System.out.print("Enter a byte: ");
        byte by = sc.nextByte();
        System.out.println("Byte: " + by);

        sc.close();
    }
}
```

Program 2: Checking Input Availability

```
import java.util.Scanner;
public class HasNext{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter input (type 'exit' to quit):");
        while (sc.hasNext()) {
            String token = sc.next();
            if (token.equalsIgnoreCase("exit")) break;
            System.out.println("You entered: " + token);
        }

        sc.close();
    }
}
```

Program 3: Using All hasNextXXX() Methods

```
import java.util.Scanner;

public class HasNextMethods {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter input:");

        System.out.println("hasNextBoolean(): " + sc.hasNextBoolean());
        System.out.println("hasNextByte(): " + sc.hasNextByte());
        System.out.println("hasNextShort(): " + sc.hasNextShort());
        System.out.println("hasNextInt(): " + sc.hasNextInt());
        System.out.println("hasNextLong(): " + sc.hasNextLong());
        System.out.println("hasNextFloat(): " + sc.hasNextFloat());
        System.out.println("hasNextDouble(): " + sc.hasNextDouble());
```

```
        sc.close();
    }
}
```

Program 4: Delimiter and Custom Split

```
import java.util.Scanner;
public class TestDelimiter{
    public static void main(String[] args) {
        String data = "apple#banana#cherry";
        Scanner sc = new Scanner(data);
        sc.useDelimiter("#"); // Custom delimiter

        while (sc.hasNext()) {
            System.out.println(sc.next());
        }

        sc.close();
    }
}
```

Program 5: Locale Usage (For Decimal Formats)

```
import java.util.*;
class LocaleScannerDemo {
    public static void main(String[] args) {
        String input = "123,45";
        Scanner sc = new Scanner(input);
        sc.useLocale(Locale.GERMANY); // Comma as decimal separator

        if (sc.hasNextDouble()) {
            double d = sc.nextDouble();
            System.out.println("Parsed double: " + d);
        }

        sc.close();
    }
}
```

Program 6: Tokens and Skipping Patterns

```
import java.util.Scanner;

public class SkipUse {
    public static void main(String[] args) {
        String input = "123 ABC";
        Scanner sc = new Scanner(input);

        System.out.println("Before skip: " + sc.next());
        sc.skip(" "); // Skip the space
        System.out.println("After skip: " + sc.next());

        sc.close();
    }
}
```

Program 7: Using findInLine() and findWithinHorizon()

```
import java.util.Scanner;

public class FindInLine {
```

```
        public static void main(String[] args) {
            Scanner sc = new Scanner("name=Jagannath age=20 gender=M");
            String name = sc.findInLine("Jagannath");
            System.out.println("Name: " + name);
            sc = new Scanner("apple banana cherry");
            String found = sc.findWithinHorizon("banana", 10); // only searches next 10
chars
            System.out.println("Found: " + found);
            sc.close();
        }
    }
```

Program 8: Accessing Scanner Metadata

```
import java.util.Scanner;

    public class ScannerMetaInfo {
        public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);

            System.out.println("Current delimiter: " + sc.delimiter());
            System.out.println("Current locale: " + sc.locale());

            sc.close();
        }
    }
```

List of All Important Methods Used

Method	Description
next(), nextLine()	Read input
nextInt(), nextDouble() etc.	Read typed input
hasNext(), hasNextInt() etc.	Check if more input exists
useDelimiter()	Change how tokens are separated
useLocale()	Use different formatting (e.g., commas)
findInLine(), findWithinHorizon()	Pattern matching
skip()	Skip text matching a pattern
delimiter(), locale()	Get current settings
close()	Close scanner

Practise String methods

1. char charAt(int index)

Description:
Returns the character at the specified index. Indexing starts from 0.
Return type: char
Example:
String name = "Jagannath";

char ch = name.charAt(3);
System.out.println(ch); // Output: a
Explanation: The character at index 3 is 'a'.

2. int codePointAt(int index)
Description:
Returns the Unicode code point of the character at the specified index.
Return type: int
Example:
String s = "𝔘"; // Unicode character
int codePoint = s.codePointAt(0);
System.out.println(codePoint); // Output: 120088

3. int codePointBefore(int index)
Description:
Returns the Unicode code point of the character before the specified index.
Return type: int
Example:
String s = "𝔘";
int codePoint = s.codePointBefore(1);
System.out.println(codePoint); // Output: 120088

4. int codePointCount(int beginIndex, int endIndex)
Description:
Returns the number of Unicode code points in the specified text range.
Return type: int
Example:
String s = "Hello";
int count = s.codePointCount(0, s.length());
System.out.println(count); // Output: 5

5. int compareTo(String anotherString)
Description:
Compares two strings lexicographically (case sensitive). Returns 0 if equal, negative if this string is lex smaller, positive if greater.
Return type: int
Example:
System.out.println("apple".compareTo("banana")); // Output: negative number
System.out.println("apple".compareTo("apple"));   // Output: 0
System.out.println("banana".compareTo("apple")); // Output: positive number

6. int compareToIgnoreCase(String str)
Description:
Compares two strings lexicographically ignoring case.
Return type: int
Example:
System.out.println("apple".compareToIgnoreCase("APPLE")); // Output: 0

7. String concat(String str)
Description:
Concatenates the specified string to this string.
Return type: String
Example:
String fullName = "Jagannath".concat(" Bhandari");
System.out.println(fullName); // Output: Jagannath Bhandari

8. boolean contains(CharSequence s)
Description:
Checks if this string contains the specified sequence.
Return type: boolean
Example:
```
String s = "Jagannath is a Software Developer";
System.out.println(s.contains("Software"));  // Output: true
System.out.println(s.contains("Engineer"));  // Output: false
```

9. boolean contentEquals(CharSequence cs)
Description:
Checks if this string equals the content of the specified CharSequence.
Return type: boolean
Example:
```
String s = "Jagannath";
StringBuffer sb = new StringBuffer("Jagannath");
System.out.println(s.contentEquals(sb));  // Output: true
```

10. boolean endsWith(String suffix)
Description:
Checks if this string ends with the specified suffix.
Return type: boolean
Example:
```
String filename = "resume.pdf";
System.out.println(filename.endsWith(".pdf"));  // Output: true
System.out.println(filename.endsWith(".doc"));  // Output: false
```

11. boolean equals(Object anObject)
Description:
Compares this string to the specified object for equality.
Return type: boolean
Example:
```
String s1 = "Jagannath";
String s2 = "jagannath";
System.out.println(s1.equals(s2));  // Output: false (case-sensitive)
```

12. boolean equalsIgnoreCase(String anotherString)
Description:
Compares strings ignoring case.
Return type: boolean
Example:
```
String s1 = "Jagannath";
String s2 = "jagannath";
System.out.println(s1.equalsIgnoreCase(s2));  // Output: true
```

13. byte[] getBytes()
Description:
Encodes the string into a byte array using platform's default charset.
Return type: byte[]
Example:
```
byte[] bytes = "Jagannath".getBytes();
System.out.println(  .util.Arrays.toString(bytes));
```

14. byte[] getBytes(Charset charset)

Description:
Encodes string into byte array using the specified charset.
Return type: byte[]
Example:
```
byte[] bytes = "Jagannath".getBytes(  .nio.charset.StandardCharsets.UTF_8);
System.out.println(  .util.Arrays.toString(bytes));
```

15. byte[] getBytes(String charsetName)
Description:
Encodes string using given charset (throws UnsupportedEncodingException).
Return type: byte[]
Example:
```
try {
    byte[] bytes = "Jagannath".getBytes("UTF-8");
    System.out.println(  .util.Arrays.toString(bytes));
} catch (  .io.UnsupportedEncodingException e) {
    e.printStackTrace();
}
```

16. void getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)
Description:
Copies characters from this string into the destination char array.
Return type: void
Example:
```
char[] buffer = new char[5];
"Jagannath".getChars(0, 5, buffer, 0);
System.out.println(  .util.Arrays.toString(buffer));  // Output: [J, a, g, a, n]
```

17. int hashCode()
Description:
Returns hash code for the string.
Return type: int
Example:
```
int hash = "Jagannath".hashCode();
System.out.println(hash);
```

18. int indexOf(int ch)
Description:
Returns the index of first occurrence of the specified character.
Return type: int
Example:
```
String s = "Jagannath";
System.out.println(s.indexOf('a'));  // Output: 1
```

19. int indexOf(int ch, int fromIndex)
Description:
Returns index of first occurrence of character after fromIndex.
Return type: int
Example:
```
String s = "Jagannath";
System.out.println(s.indexOf('a', 2));  // Output: 3
```

20. int indexOf(String str)
Description:
Returns index of first occurrence of substring.

Return type: int
Example:
String s = "Jagannath";
System.out.println(s.indexOf("nat")); // Output: 5

21. int indexOf(String str, int fromIndex)
Description:
Returns index of substring after fromIndex.
Return type: int
Example:
String s = "JagannathJagannath";
System.out.println(s.indexOf("nat", 6)); // Output: 14

22. boolean isEmpty()
Description:
Returns true if string length is 0.
Return type: boolean
Example:
String s1 = "";
System.out.println(s1.isEmpty()); // Output: true
String s2 = " ";
System.out.println(s2.isEmpty()); // Output: false

23. int lastIndexOf(int ch)
Description:
Returns index of last occurrence of character.
Return type: int
Example:
String s = "Jagannath";
System.out.println(s.lastIndexOf('a')); // Output: 6

24. int lastIndexOf(int ch, int fromIndex)
Description:
Returns last occurrence of character before or at fromIndex.
Return type: int
Example:
String s = "Jagannath";
System.out.println(s.lastIndexOf('a', 5)); // Output: 3

25. int lastIndexOf(String str)
Description:
Returns last index of substring.
Return type: int
Example:
String s = "JagannathJagannath";
System.out.println(s.lastIndexOf("nat")); // Output: 14

26. int lastIndexOf(String str, int fromIndex)
Description:
Returns last index of substring before fromIndex.
Return type: int
Example:
String s = "JagannathJagannath";
System.out.println(s.lastIndexOf("nat", 10)); // Output: 5

27. int length()
Description:
Returns length of the string.
Return type: int
Example:
String s = "Jagannath";
System.out.println(s.length()); // Output: 9

28. boolean matches(String regex)
Description:
Tests if string matches regex pattern.
Return type: boolean
Example:
String email = "user@example.com";
System.out.println(email.matches("^.+@.+\\..+$")); // Output: true

29. boolean regionMatches(int toffset, String other, int ooffset, int len)
Description:
Tests if substring of this string equals substring of another string.
Return type: boolean
Example:
String s1 = "Jagannath";
String s2 = "JaganXYZ";
System.out.println(s1.regionMatches(0, s2, 0, 5)); // Output: true

30. boolean regionMatches(boolean ignoreCase, int toffset, String other, int ooffset, int len)
Description:
Tests if substrings match with optional case ignore.
Return type: boolean
Example:
String s1 = "Jagannath";
String s2 = "jAGAnxyz";
System.out.println(s1.regionMatches(true, 0, s2, 0, 5)); // Output: true

31. String replace(char oldChar, char newChar)
Description:
Replaces all occurrences of oldChar with newChar.
Return type: String
Example:
 String s = "Jagannath";
String replaced = s.replace('a', 'o');
System.out.println(replaced); // Output: Jogonnoth

32. String replace(CharSequence target, CharSequence replacement)
Description:
Replaces occurrences of target sequence with replacement.
Return type: String
Example:
String s = "Jagannath is a Software Developer";
String replaced = s.replace("Software", " ");
System.out.println(replaced); // Output: Jagannath is a   Developer

33. String replaceAll(String regex, String replacement)
Description:

Replaces all substrings matching regex with replacement.
Return type: String
Example:
String s = "My phone is 9876543210";
String replaced = s.replaceAll("\\d", "*");
System.out.println(replaced); // Output: My phone is **********

34. String replaceFirst(String regex, String replacement)
Description:
Replaces the first substring matching regex with replacement.
Return type: String
Example:
String s = "one one one";
String replaced = s.replaceFirst("one", "two");
System.out.println(replaced); // Output: two one one

35. String[] split(String regex)
Description:
Splits string around matches of the regex.
Return type: String[]
Example:
String s = "apple,banana,orange";
String[] fruits = s.split(",");
for(String fruit : fruits) {
    System.out.println(fruit);
}
// Output:
// apple
// banana
// orange

36. String[] split(String regex, int limit)
Description:
Splits string into a limited number of substrings.
Return type: String[]
Example:
String s = "a,b,c,d";
String[] parts = s.split(",", 2);
System.out.println(.util.Arrays.toString(parts)); // Output: [a, b,c,d]

37. boolean startsWith(String prefix)
Description:
Checks if string starts with prefix.
Return type: Boolean
Example:
String s = "Jagannath";
System.out.println(s.startsWith("Jag")); // Output: true

38. boolean startsWith(String prefix, int toffset)
Description:
Checks if substring starting at toffset starts with prefix.
Return type: boolean
Example:

String s = "Jagannath";
System.out.println(s.startsWith("ann", 3)); // Output: true
39. CharSequence subSequence(int beginIndex, int endIndex)
Description:
Returns subsequence from beginIndex (inclusive) to endIndex (exclusive).
Return type: CharSequence
Example:
String s = "Jagannath";
CharSequence sub = s.subSequence(1, 4);
System.out.println(sub); // Output: aga

40. String substring(int beginIndex)
Description:
Returns substring from beginIndex to end.
Return type: String
Example:
String s = "Jagannath";
String sub = s.substring(3);
System.out.println(sub); // Output: annath

41. String substring(int beginIndex, int endIndex)
Description:
Returns substring from beginIndex (inclusive) to endIndex (exclusive).
Return type: String
Example:
 String s = "Jagannath";
String sub = s.substring(3, 6);
System.out.println(sub); // Output: ann
42. char[] toCharArray()
Description:
Converts string into a char array.
Return type: char[]
Example:
String s = "Jagannath";
char[] chars = s.toCharArray();
System.out.println(.util.Arrays.toString(chars)); // Output: [J, a, g, a, n, n, a, t, h]

43. String toLowerCase()
Description:
Returns string converted to lower case.
Return type: String
Example:
String s = "Jagannath";
System.out.println(s.toLowerCase()); // Output: jagannath

44. String toLowerCase(Locale locale)
Description:
Returns string converted to lower case using specified locale.
Return type: String
Example:
String s = "İstanbul";
System.out.println(s.toLowerCase(new .util.Locale("tr"))); // Output: istanbul (Turkish locale)

45. String toUpperCase()

Description:
Returns string converted to upper case.
Return type: String
Example:
String s = "Jagannath";
System.out.println(s.toUpperCase()); // Output: JAGANNATH

46. String toUpperCase(Locale locale)
Description:
Returns string converted to upper case using specified locale.
Return type: String
Example:
String s = "istanbul";
System.out.println(s.toUpperCase(new .util.Locale("tr"))); // Output: İSTANBUL

47. String trim()
Description:
Removes leading and trailing spaces (ASCII whitespace).
Return type: String
Example:
String s = " Jagannath ";
System.out.println("[" + s.trim() + "]"); // Output: [Jagannath]

48. String strip()
Description:
Removes leading and trailing whitespace (Unicode aware, 11+).
Return type: String
Example:
String s = " Jagannath ";
System.out.println("[" + s.strip() + "]"); // Output: [Jagannath]

49. String stripLeading()
Description:
Removes leading whitespace only.
Return type: String
Example:
String s = " Jagannath ";
System.out.println("[" + s.stripLeading() + "]"); // Output: [Jagannath]

50. String stripTrailing()
Description:
Removes trailing whitespace only.
Return type: String
Example:
String s = " Jagannath ";
System.out.println("[" + s.stripTrailing() + "]"); // Output: [Jagannath]

51. boolean isBlank()
Description:
Returns true if string is empty or contains only whitespace (11+).
Return type: boolean
Example:
System.out.println(" ".isBlank()); // Output: true
System.out.println("abc".isBlank()); // Output: false

52. String repeat(int count)
Description:
Returns string repeated count times (11+).
Return type: String
Example:
System.out.println("Ja".repeat(3)); // Output: JaJaJa

53. Stream<String> lines()
Description:
Returns a stream of lines extracted from the string (11+).
Return type: Stream<String>
Example:
String s = "Jagannath\nBhandari\nDeveloper";
s.lines().forEach(System.out::println);
// Output:
// Jagannath
// Bhandari
// Developer

54. String formatted(Object... args)
Description:
Returns a formatted string using String.format.
Return type: String
Example:
String template = "My name is %s and I am %d years old.";
System.out.println(template.formatted("Jagannath", 20));
// Output: My name is Jagannath and I am 20 years old.

User Defined Method:

Q. What is a user-defined method?
Answer: A method which is created by the programmer is called as user defined method.

Q. Why do we need user defined method when we already have predefined method?
Answer: Whenever predefined method doesnot fulfill the requirement of project we need to design user defined methods to fulfill the requirement.
Above to that, user defined method reduces code duplication,improves code readability and maintainability,helps in debugging and testing,makes complex programs modular.
Syntax of a User-Defined Method:

```
return_type methodName(parameters) {
        // method body
    }
```

e.g:

```
public class VotingEligibility {

        // User-defined method
        public static void checkEligibility(int age) {
            if (age >= 18) {
                System.out.println("You are eligible to vote.");
            } else {
                System.out.println("You are not eligible to vote.");
            }
        }

        public static void main(String[] args) {
            int userAge = 20;
```

```
            // Calling user-defined method
            checkEligibility(userAge);
        }
    }
```

Q. How many user defined method can be created?
Answer: Depends on requirement.

Q. How many times we can call a user defined method?
Answer: Depends on requirement.

Q. How do we call user defined methods to execute?
Answer: We call user defined methods by their name, from main method.

Q. Can we call predefined method inside user defined method?
Answer: Yes predefined method can be called inside user defined method.

Q. Can we call one user defined method from another user defined method?
Answer: Yes.

//WAP to enter any two numbers and display their sum using user defined method

```
import java.util.Scanner;
class Test1
{
   public static void findSum(int a,int b){
   System.out.println("Sum of "+a+" and "+b+" : "+(a+b));
   }
public static void main(String []args){
Scanner sc = new Scanner(System.in);
System.out.println("Enter 1st number: ");
int a = sc.nextInt();
System.out.println("Enter 2nd number: ");
int b = sc.nextInt();
findSum(a,b);
}
}
```

Q1.Create a class BillingSystem that contains two static methods:
getGST() – returns a fixed GST rate (like 18%).
calculateTotalWithGST() – calculates and prints the total price including GST.
Access getGST() directly inside calculateTotalWithGST().
Solution:

```
public class BillingSystem {

        // Static user defined method returning fixed GST rate
        public static int getGST() {
            return 18; // GST rate in percentage
        }

        // Static user defined  method to calculate total with GST
        public static void calculateTotalWithGST(double basePrice) {
            int gstRate = getGST(); // Access getGST directly
            double totalAmount = basePrice + (basePrice * gstRate / 100);
            System.out.println("Base Price: " + basePrice);
            System.out.println("GST Rate: " + gstRate + "%");
            System.out.println("Total Price (including GST): " + totalAmount);
```

```
    }

    // Main method to test
    public static void main(String[] args) {
        calculateTotalWithGST(1000); // Sample base price
    }
}
```

Output:

Base Price: 1000.0
GST Rate: 18%
Total Price (including GST): 1180.0

Q2. Create a class OnlineLibrary with:
A static method getLibraryName() that returns a library name.
A non-static method displayWelcomeMessage() that prints a welcome message including the library name.
Access getLibraryName() directly inside displayWelcomeMessage().
Solution:

```
import java.util.Scanner;

public class OnlineLibrary {

    // Static method to return the library name
    public static String getLibraryName() {
        return "Digital Knowledge Hub";
    }

    // Non-static method that prints a welcome message
    public void displayWelcomeMessage(String userName) {
        String libName = getLibraryName(); // Access static method directly
        System.out.println("Welcome " + userName + " to " + libName + "!");
        System.out.println("Explore thousands of books and digital resources.");
    }

    // Main method
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String userName = sc.nextLine();

        // Create object to call non-static method
        OnlineLibrary ol = new OnlineLibrary();
        ol.displayWelcomeMessage(userName);

        sc.close();
    }
}
```

Q3. Create a class TemperatureConverter with:
A non-static method convertToFahrenheit(double celsius) that returns the Fahrenheit value.

A static method printConvertedTemperature() that creates an object of TemperatureConverter and calls the non-static method to print the result.
Solution:

```
public class TemperatureConverter {

        // Non-static method to convert Celsius to Fahrenheit
        public double convertToFahrenheit(double celsius) {
            return (celsius * 9 / 5) + 32;
        }

        // Static method that creates object and calls the non-static method
        public static void printConvertedTemperature(double celsiusInput) {
            TemperatureConverter converter = new TemperatureConverter();
            double fahrenheit = converter.convertToFahrenheit(celsiusInput);
            System.out.println("Celsius: " + celsiusInput);
            System.out.println("Fahrenheit: " + fahrenheit);
        }

        // Main method for testing
        public static void main(String[] args) {
            printConvertedTemperature(37); // Example input
        }
    }
```

Q4. Create a class EmployeePayroll with:
A non-static method calculateHRA(double salary) that returns HRA.
A non-static method calculateNetSalary(double salary) that directly calls calculateHRA() and calculates the total salary including HRA.
Solution:

```
public class EmployeePayroll {

        // Non-static method to calculate HRA (e.g., 20% of salary)
        public double calculateHRA(double salary) {
            return salary * 0.20; // 20% HRA
        }

        // Non-static method to calculate net salary including HRA
        public double calculateNetSalary(double salary) {
            double hra = calculateHRA(salary); // Call to non-static method
            return salary + hra;
        }

        public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);
            EmployeePayroll payroll = new EmployeePayroll();

            System.out.print("Enter basic salary: ");
            double salary = sc.nextDouble();

            double hra = payroll.calculateHRA(salary);
            double netSalary = payroll.calculateNetSalary(salary);
```

```
        System.out.println("HRA (20%): " + hra);
        System.out.println("Net Salary (Basic + HRA): " + netSalary);

        sc.close();
    }
}
```

Q5. Create two classes:
Class MathUtility with a static method getSquare(int num) that returns the square.
Class MathApplication with a static method showResult() that accesses MathUtility.getSquare() using the class name.
Solution:

Q6. Create two classes:
Class CurrencyConverter with a static method getDollarRate() returning conversion rate.
Class TravelAgency with a non-static method displayConversionRate() that accesses CurrencyConverter.getDollarRate() using the class name.

Q7. Create two classes:

Class BankAccount with a non-static method getBalance() returning account balance.
Class BankSystem with a static method printBalance() that creates an object of BankAccount and accesses getBalance().

Q8. Create two classes:
Class Student with a non-static method getStudentDetails() that returns name and marks.
Class SchoolSystem with a non-static method printStudentInfo() that creates an object of Student and accesses getStudentDetails().

Practice 2
Q1. Write a program in Java to accept a String from the user. Pass the String to a method which displays number of the consonants present in the String.

Sample Input: computer
Sample Output:
c
m
p
t
r
No. of consonants in string: 5

Q2. Write a program in Java to accept a String from the user. Pass the

String to a method which displays the first character of each word after changing the case (lower to upper and vice versa).
Sample Input: Naresh i Technologies
Sample Output:
n
I
t

Q3. Write a program in Java to accept a String from the user. Pass the String to a method which displays the first character of each word.
Sample Input : Understanding Core Java
Sample Output:
U
C
A

Q4.Instagram restricts who can view stories depending on user relationships and privacy settings.
You are required to design a method that checks if a user can view another user's story based on the following conditions:

The user must be following the person.
If the account is private, the viewer must be a close friend to see the story.

Task:
Create a class InstagramUser with three boolean variables: isFollowing, isPrivateAccount, and isCloseFriend.
Define a method canViewStory() that returns:
"Can view" if the user is following and either the account is not private or they are a close friend.
"Cannot view" otherwise.
Solution:

```
import java.util.Scanner;

        public class InstagramUser {

            boolean isFollowing;
            boolean isPrivateAccount;
            boolean isCloseFriend;

            // Method to check if the user can view story
            public String canViewStory() {
                if (isFollowing && (!isPrivateAccount || isCloseFriend)) {
                    return "Can view";
                } else {
                    return "Cannot view";
                }
            }

            public static void main(String[] args) {
                Scanner sc = new Scanner(System.in);

                InstagramUser user = new InstagramUser();
```

```
        System.out.print("Is the user following? (true/false): ");
        user.isFollowing = sc.nextBoolean();

        System.out.print("Is the account private? (true/false): ");
        user.isPrivateAccount = sc.nextBoolean();

        System.out.print("Is the user a close friend? (true/false): ");
        user.isCloseFriend = sc.nextBoolean();

        System.out.println(user.canViewStory());

        sc.close();
    }
}
```

Q5. Design Friend Suggestion System Module.
Instagram suggests friends based on mutual connections and account verification status. Verified users are prioritized for suggestions.

Task:
Create a class User with:
int mutualFriends
boolean isVerified
Write a method getSuggestionLevel() that returns:
- "Strong Suggestion" if mutualFriends ≥ 10 and the user is verified.
- "Moderate Suggestion" if mutualFriends is between 1 and 9.
- "Low Suggestion" if mutualFriends = 0 or the user is not verified.
Solution:

```
import java.util.Scanner;

public class User {
    int mutualFriends;
    boolean isVerified;

    // Method to get suggestion level
    public String getSuggestionLevel() {
        if (mutualFriends >= 10 && isVerified) {
            return "Strong Suggestion";
        } else if (mutualFriends >= 1 && mutualFriends <= 9) {
            return "Moderate Suggestion";
        } else {
            return "Low Suggestion";
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        User user = new User();
```

```
            System.out.print("Enter number of mutual friends: ");
            user.mutualFriends = sc.nextInt();

            System.out.print("Is the user verified? (true/false): ");
            user.isVerified = sc.nextBoolean();

            System.out.println(user.getSuggestionLevel());

            sc.close();
        }
    }
```

Sample Output:

Enter number of mutual friends: 12
Is the user verified? (true/false): true
Strong Suggestion

Q6. Delivery Charge Calculation Module
Description:
Swiggy applies delivery charges based on distance, but users with a Swiggy One membership get free delivery.
Task:
Create a class UserOrder with:
int distanceInKm
boolean hasMembership

Write a method getDeliveryCharge() that:
- Returns ₹0 if the user has membership.
- If not a member:
₹0 for ≤ 2 km
₹20 for 3–5 km
₹50 for > 5 km

Q7. Restaurant Availability Check Module in Swiggy
Description:
Restaurants are only visible to users if the restaurant is open, delivers to the user's location, and the user's location is within a 8 km radius.

Task:
Create a class Restaurant with:
boolean isOpen
boolean deliversToUser
int userDistance

Write a method isAvailable() that returns:
- "Available" if all conditions are satisfied.
- "Not Available" otherwise.

Q8. Coupon Application Eligibility Module in Swiggy
Description:
Coupons can be applied to a Swiggy order only if:
The total order amount is greater than ₹200.
The user is logged in.(Take user ID fixed by developer)

The coupon code is valid.(Take coupon code fixed by developer)

Task:
Create a class Coupon with:
int totalAmount
boolean isLoggedIn
boolean isValidCoupon

Write a method applyCoupon() that returns:
- "Coupon Applied" if all conditions are met.
- "Not Eligible" with a specific reason otherwise.

Q9. A bank app checks loan eligibility based on credit score, salary, and age. The minimum requirements are:
Credit score must be above 750.
Monthly salary must exceed ₹50,000.
Age should be between 21 and 60.

Task:
Create a class Customer with:
int creditScore
int salary
int age

Write a method isEligibleForLoan() that returns:
- "Eligible" if all conditions are met.
- "Not Eligible" with a specific reason (e.g., low credit score, insufficient salary, or invalid age).

Q10. iMobile provides different interest rates based on the account type. For senior citizens, there's an additional interest benefit (except for Current accounts).
Task:
Create a class BankAccount with:
String accountType (can be "Savings", "Fixed", "Current")
boolean isSeniorCitizen

Write a method getInterestRate() using a switch that returns:
- "Interest Rate: 5.5%" for Savings if senior, else "4.5%"
- "7.0%" for Fixed if senior, else "6.0%"
- "5.0%" for Current (same for all)
- "Invalid Account Type" for any other input
Solution:

```
import java.util.Scanner;

        public class BankAccount {
            String accountType;
            boolean isSeniorCitizen;

            // Method to get interest rate string using switch
            public String getInterestRate() {
                String type = accountType.toLowerCase();

                switch (type) {
                    case "savings":
```

```
                if (isSeniorCitizen) {
                    return "Interest Rate: 5.5%";
                } else {
                    return "Interest Rate: 4.5%";
                }

            case "fixed":
                if (isSeniorCitizen) {
                    return "Interest Rate: 7.0%";
                } else {
                    return "Interest Rate: 6.0%";
                }

            case "current":
                return "Interest Rate: 5.0%";

            default:
                return "Invalid Account Type";
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BankAccount account = new BankAccount();

        System.out.print("Enter account type (Savings/Fixed/Current): ");
        account.accountType = sc.nextLine().trim();

        System.out.print("Is the account holder a senior citizen? (true/false): ");
        account.isSeniorCitizen = sc.nextBoolean();

        System.out.println(account.getInterestRate());

        sc.close();
    }
}
```

Static Method:

Q. What is a static method?

Answer: A method which is created using 'static' keyword is called as static method.

Q. Why do we need static method?

Answer: When we have to access method without using object then we must use static method.

Q. How do we acess static method?

Answer: By using class name or directly.

Q. When do we use class name to access static method?

Answer: Whenever we access static method of one class from another class.

Q. How many static methods can be declared in a program?

Answer: Based on requirement.

Q. Can we access NSM from SM?
Answer: Yes by using object or object reference.

Q. Where are the static methods loaded?
Answer: SM are loaded inside static pool area by class loader during class loading(Execution Phase).

Q. How many copy of memory does a static method have?
Answer: single copy of memory.

Q. How many times static method is loaded into the memory?
Answer: Only once throughout the execution.

Non- Static Method:

Q. What is a non static method?
Answer: A method which is created without using 'static' keyword is called as non-static method.

Q. Why do we need non-static method?
Answer: When we have to access method using object then we must use non-static method.

Q. How do we acess non static method?
Answer: By using object or object reference.

Q. When do we access non static method directly?
Answer: Whenever we have to access one non static method from another non static method in same class we access directly.

Q. How many non static methods can be declared in a program?
Answer: Based on requirement.

Q. Can we access NSM from NSM?
Answer: Yes we can access directly if its in same class.

Q. Where are the non static methods loaded?
Answer: NSM are loaded inside object in the heap memory whenever an object is created.

Q. How many copy of memory does a non static method have?
Answer: multiple copy of memory, depending on number of object created.

Q. How many times non static method is loaded into the memory?
Answer: depends on creation of object.

Flipkart Order Management System
Problem Statement:
Develop an Order Management System for Flipkart that allows users to place, view, and cancel orders. The system should also track the total number of orders placed.

Features:
- Customers can place orders by specifying product details.
- Customers can view order details.
- Customers can cancel an order before it is shipped.

- The system should track the total number of orders.
- Calculate the total cost of the order dynamically.

Attributes (Variables):
Static Variable:
totalOrders: Tracks the total number of orders.

Instance Variables:
orderId: Unique ID for each order.
productName: Name of the product ordered.
price: Price per unit of the product.
quantity: Number of units purchased.
status: Status of the order (Confirmed, Canceled).

- Local Variable:
totalCost: Holds the computed total order cost (price × quantity).

Operations (Methods):
- placeOrder(): Places a new order and increments totalOrders.
- cancelOrder(): Cancels the order and updates its status.
- calculateTotalCost(): Computes total cost dynamically.
- getTotalOrders(): Returns the total number of orders placed (static method).
- displayOrderDetails(): Displays order information.

Additional Operations:
- Apply discount coupons while placing an order.
- Allow updating order quantity before confirmation.
- Provide estimated delivery time.

Food Delivery App
You are designing a system for a food delivery app where users can browse food items. Each food item has a name, price, and category (e.g., "Starter", "Main Course", "Dessert").
Users should be able to view and update the details of each item.

Task:
Create a class FoodItem with the following private fields:
String itemName — stores the name of the food item (e.g., "Burger").
double price — stores the price of the item (e.g., 120.50).
String category — stores the category of the item (e.g., "Main Course").

Implement the following:
Setter Methods: Use the this keyword to assign values to private fields, ensuring no conflict with parameter names.
Getter Methods: Allow other classes to retrieve values of these fields.

Write a main class to:
- Create 3 food items (FoodItem objects).
- Use setter methods to set their details.
- Display the details using getter methods.
- Update the price of one food item and print the updated details.

Two Categories of methods based on returning value:
- Method with data type as return type:

Whenever output of an operation is required as input for another operation

then we must use method with data type as return type.
e.g: **WAP to find the area of a circle whose diameter is 100.0cm.**
Area of circle is Math.PI*r*r.

```
class Circle{
double diameter;
double area;
public void findArea(double radius){
System.out.println("Area: "+Math.PI*radius*radius);
}
public double findRadius(double diameter){
return diameter/2.0;
}
}
class TestCircle{
public static void main(String []args){
Scanner sc = new Scanner(System.in);
System.out.println("Enter the diameter of circle: ");
Circle c1 = new Circle();
c1.diameter = sc.nextDouble();
double radius = c1.findRadius(diameter);
c1.findArea(radius);
}
}
```

Practise:
Q1.
WAP for below requirement:
i. 3 cars of same brand moving at different speed and travelled for different amount of time.
ii. Find the distance travelled by each car and print all the details of car.
Take class name as Car.Take 1 SV for car brand, 2 nsv for speed and time.
Create objects to assign values to NSV.
Create a method to findDistance travelled by car.
Solution:

```
public class Car {
static String brand = "Hyundai";  // Static variable (SV)

int speed;  // Non-static variable (NSV)
int time;   // Non-static variable (NSV)

// Method to calculate distance
public int findDistance() {
return speed * time;
}

// Method to display car details
public void displayDetails(int carNumber) {
System.out.println("Car " + carNumber + " Details:");
System.out.println("Brand: " + brand);
System.out.println("Speed: " + speed + " km/h");
System.out.println("Time: " + time + " hours");
System.out.println("Distance Travelled: " + findDistance() + " km");
System.out.println("----------------------------");
}
```

```
public static void main(String[] args) {
// Car 1
Car car1 = new Car();
car1.speed = 60;
car1.time = 2;

// Car 2
Car car2 = new Car();
car2.speed = 80;
car2.time = 3;

// Car 3
Car car3 = new Car();
car3.speed = 100;
car3.time = 1;

// Display all car details
car1.displayDetails(1);
car2.displayDetails(2);
car3.displayDetails(3);
}
}
```

Q2.
International Cricket Council is the global governing body for cricket.The ICC governs and administrates the game and works with its 108 members to grow the sport. The ICC is also responsible for the staging of all ICC Events.
The ICC presides over the ICC Code of Conduct, playing conditions, the Decision Review System and other ICC regulations.Recently ICC came up with a project for generating the Men's Test Batting Rankings
Create a class BatsMan with the following specifications:
Create variables as per requirement.

Create Players to initialize:
Team,Player'sName,MatchesPlayed,TotalRuns,NoOfBallsFaced
void calculateAverage(): compute & display the averageScore by dividing no. of runs with no. of matches played till date.

void strikeRate(): compute and initialize strike rate of player
Strike rate = (Runs scored / Balls faced) x 100

void calculateRating(): compute and initialize the rating
void displayDetails() to display the BatsMan details in the following format:

Team	Name	MatchesPlayed	TotalRuns	NoOfBallsFaced	StrikeRate
INDIA	V KOHLI	113	8848	15924	55.56

Create a main class to test your logic.

Q3. Write a program in Java to input a character. Find and display the **next 5 characters in the ASCII table.**

Q4.WAP for the below requirement:
Program for a rectangle
Find the width of the rectangle based on the given area and length from the user
Area and length will be given in integer

Q5. A man is walking at a speed of 5m/sec, crosses the bridge of length 120m. Find the time taken to cross the bridge in minutes. Design two different methods to solve the problem.

Q7. WAP for the below requirement:
i. Create 3 classes
ii. 1st class contains SV,NSV,SM NSM. Print the SV in SM, NSV in NSM
iii. 2nd class contains SM which will call SM of 1st class, Create NSM which will call the NSM of 1st class.
iv. 3rd class contains main method which will call SM and NSM of 2nd class.

Q8. Create a class. Create NSV and NSM. Create a LV inside NSM with the same name used as NSV. Access the NSV and LV separately inside the NSM. Call the NSM from the main method.

Q9.
Telangana Electricity Board charges from their consumers according to the units consumed per month.The amount is calculated as per the tariff given below:

Units Consumed	Charge
Upto 100 units	₹5.50 per unit
For next 200 units	₹6.50 per unit
For next 300 units	₹7.50 per unit
For next 600 units	₹8.50 per unit

WAP to initialize conusmer's name, comsumer's number and the units consumed.The program displays the floowing information after calculating the amount.

Money Receipt
Consumer's Name :
Consumer's Number:
Units Consumed :
Amount to be paid:
Solution:

```
import java.util.Scanner;

        public class ElectricityBill {
            String consumerName;
            String consumerNumber;
            int units;

            // Method to calculate total amount based on units
            public double calculateAmount() {
                double amount = 0;

                if (units <= 100) {
                    amount = units * 5.50;
                } else if (units <= 300) {
                    amount = (100 * 5.50) + ((units - 100) * 6.50);
                } else if (units <= 600) {
                    amount = (100 * 5.50) + (200 * 6.50) + ((units - 300) * 7.50);
                } else {
                    amount = (100 * 5.50) + (200 * 6.50) + (300 * 7.50) + ((units - 600) * 8.50);
                }
```

```
        return amount;
    }

    // Method to display the money receipt
    public void printReceipt() {
        System.out.println("---------- Money Receipt ----------");
        System.out.println("Consumer's Name   : " + consumerName);
        System.out.println("Consumer's Number : " + consumerNumber);
        System.out.println("Units Consumed    : " + units);
        System.out.printf ("Amount to be paid : ₹%.2f\n", calculateAmount());
        System.out.println("-----------------------------------");
    }

    // Main method
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ElectricityBill bill = new ElectricityBill();

        System.out.print("Enter consumer's name: ");
        bill.consumerName = sc.nextLine();

        System.out.print("Enter consumer's number: ");
        bill.consumerNumber = sc.nextLine();

        System.out.print("Enter units consumed: ");
        bill.units = sc.nextInt();

        bill.printReceipt();

        sc.close();
    }
}
```


You are developer and you got a project in which client has asked to develop a payment application. This application will be embedded into the e-commerce domain like Flipkart,Amazon,SnapDeal,Meesho,AJIO,Myntra,AliBaba etc.
Earlier days e-commerce platforms were having option to make payment. Payment was getting done only using card or COD.
Your job is to redesign the complete application module to let the users make payments in various ways:
- card,COD,UPI,net banking,payLater.

Q. WAP to create an array and store 10 different numbers and do the following:
i) display the even numbers separately
ii) display odd numbers separately
iii) display the prime numbers separately
iv) display the sum of all the numbers
v) display the sum of all the even numbers
vi) display the sum of all the odd numbers
vii) display all the positive numbers
viii) display all the negative numbers
Create 8 different methods and pass the array to each method and perform the operation.

Bonus:

1. Encapsulation

Real-Time Example: Banking Application – Hiding Account Details

In a banking app, users shouldn't be allowed to directly change the balance — only through controlled methods.

```
class BankAccount {
        private double balance = 10000;  // private variable (data hiding)

        // Public method to safely access balance
        public double getBalance(String pin) {
            if (pin.equals("1234")) return balance;
            else return 0;
        }

        public void deposit(double amount) {
            if (amount > 0) {
                balance += amount;
            }
        }
    }

    public class TestEncapsulation {
        public static void main(String[] args) {
            BankAccount account = new BankAccount();
            account.deposit(5000);
            System.out.println("Your balance: $" + account.getBalance("1234"));  // Secure
access
        }
    }
```

Key Point: Internal data (balance) is hidden. Access is controlled via methods.

2. Abstraction

Real-Time Example: Google Maps –

User doesn't see routing logic, only uses the interface

In software, abstraction means hiding complex logic and exposing only essential functions.

```
abstract class NavigationService {
        abstract void getDirections(String from, String to);
    }

    class GoogleMaps extends NavigationService {
        @Override
        void getDirections(String from, String to) {
            // Abstracts real GPS, routing, API, etc.
            System.out.println("Showing fastest route from " + from + " to " + to);
        }
    }

    public class TestAbstraction {
        public static void main(String[] args) {
            NavigationService map = new GoogleMaps();
            map.getDirections("Hyderabad", "Chennai");
        }
    }
```

Key Point: Complex logic (GPS, real-time traffic, etc.) is hidden behind an abstract method.

3. Inheritance

Real-Time Example: E-commerce Platform – Product categories inherit from a common class

In Flipkart/Amazon, all products share some base features (name, price, etc.) but also have specific features.

```
class Product {
        String name;
        double price;

        void display() {
            System.out.println(name + " - ₹" + price);
        }
    }

    class Mobile extends Product {
        String brand;

        void showDetails() {
            display();
            System.out.println("Brand: " + brand);
        }
    }

    public class TestInheritance {
        public static void main(String[] args) {
            Mobile m = new Mobile();
            m.name = "iPhone 15";
            m.price = 79999;
            m.brand = "Apple";
            m.showDetails();
        }
    }
```

Key Point: Mobile inherits common properties from Product.

4. Polymorphism

Real-Time Example: Payment Gateway – One method behaves differently based on payment type

In apps like Paytm, the same method processPayment() behaves differently for UPI, CreditCard, Wallet, etc.

```
class Payment {
        void processPayment() {
            System.out.println("Processing generic payment...");
        }
    }

    class CreditCardPayment extends Payment {
        void processPayment() {
            System.out.println("Processing credit card payment...");
        }
    }

    class UpiPayment extends Payment {
        void processPayment() {
```

```
        System.out.println("Processing UPI payment...");
    }
}

public class TestPolymorphism {
    public static void main(String[] args) {
        Payment p;

        p = new CreditCardPayment();  // Polymorphic behavior
        p.processPayment();

        p = new UpiPayment();         // Polymorphic behavior
        p.processPayment();
    }
}
```

Key Point: One reference (Payment p) can point to multiple objects and call the right method at runtime.